LoRaWAN technology: An overview

Laura Belli, *Ph.D.*

University of Parma





UNIVERSITÀ
DI PARMA

April 23, 2026

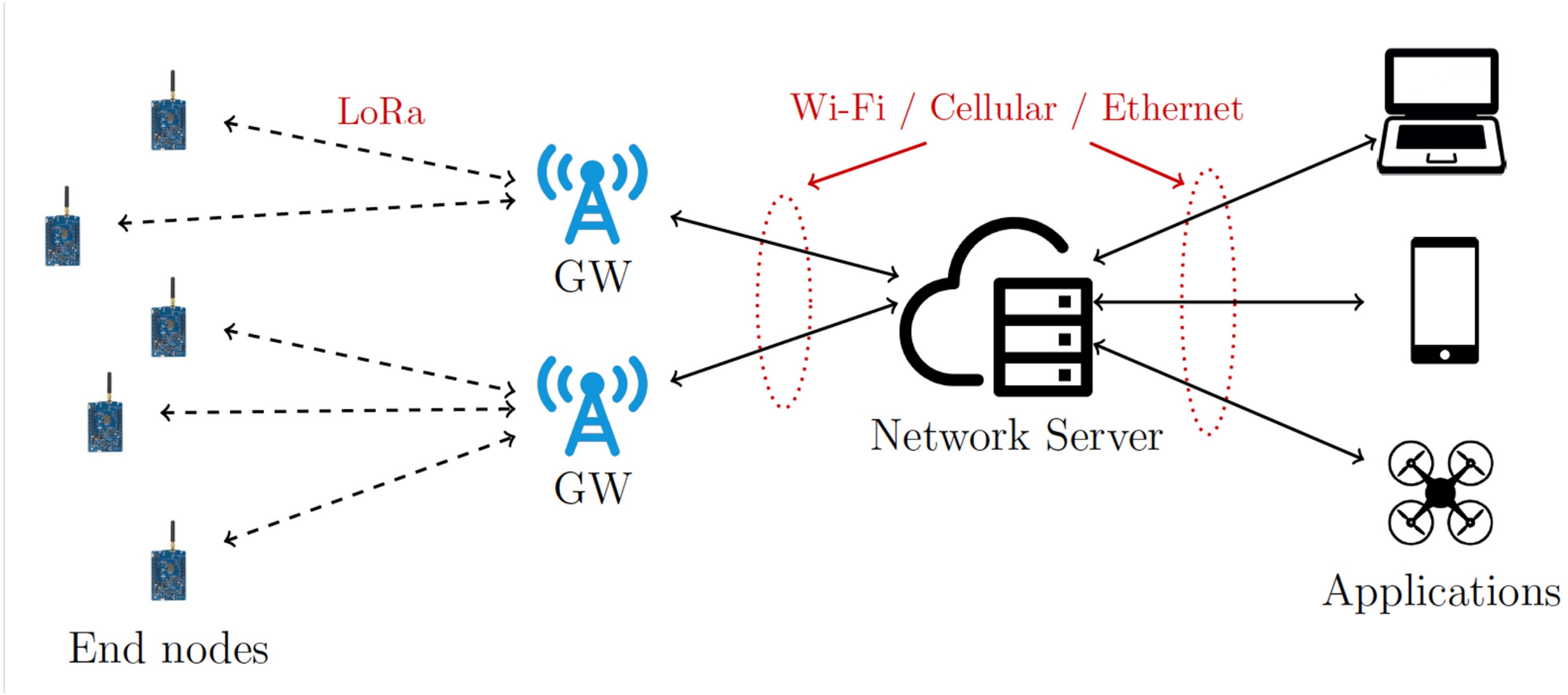
Internet of Things, Communication Engineering Course

Agenda

- LoRaWAN concepts recap
- LoRaWAN Gateways components
- The Things Network (TTN)
- ChirpStack
- LoRaWAN Devices Configuration

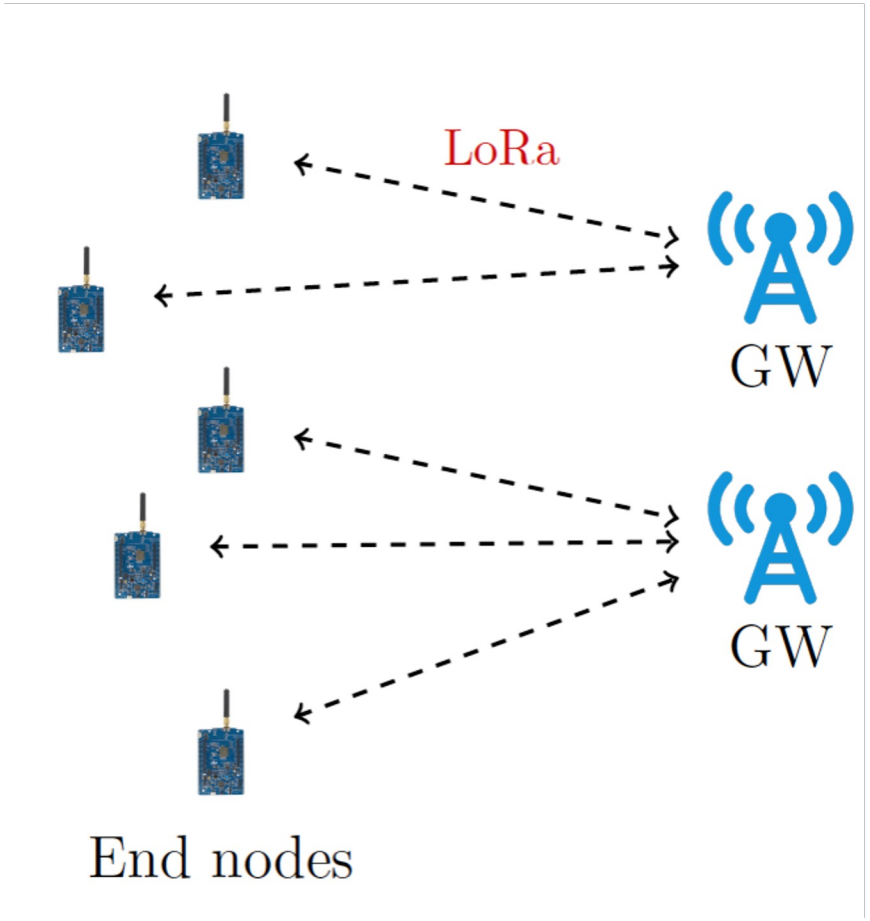
LoRaWAN recap – Network Structure

The structure of a LoRaWAN Network is a “star of stars”



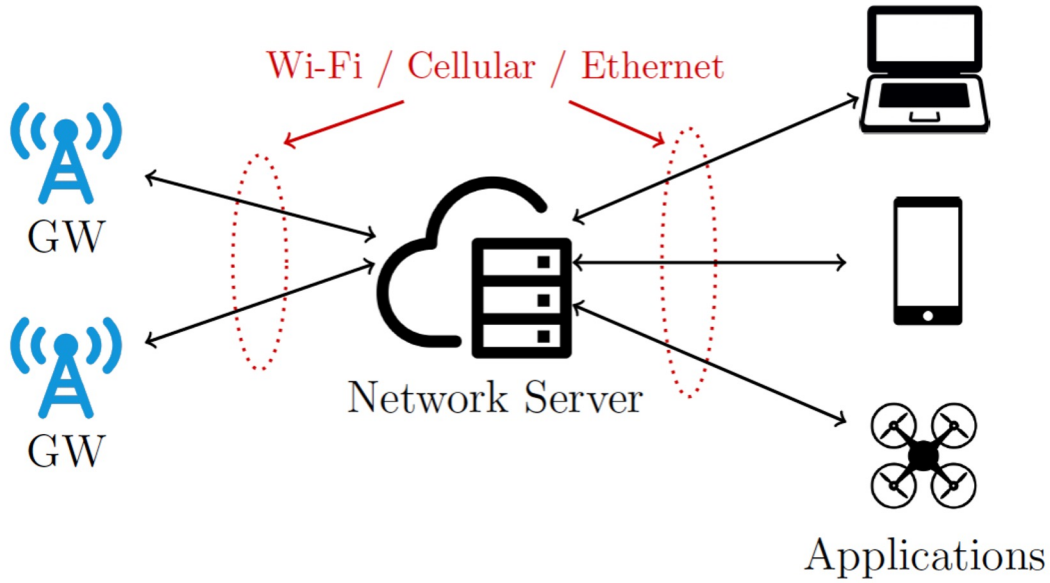
LoRaWAN recap – Network Structure

- End nodes are IoT sensors or actuators
  - They generate data (uplink messages) and can receive data (downlink messages)
  - They have to join the network before sending data
- Gateways:
  - They receive data from end nodes
  - Perform integrity checks (e.g.: Cyclic Redundancy Check CRC)
  - Forward data to the Network Server through IP packets
  - Gateways don't decrypt payloads!



LoRaWAN recap – Network Structure

- Network Server:
  - Handles join requests, downlinks, data rates and duplicated frames
  - Doesn't decrypt payloads!
- Application Server:
  - Receives packets from the Network Server
  - Schedules responses for the end nodes
  - Decrypts payloads
  - Can forward data to external entities



LoRaWAN recap – End Nodes

- LoRaWAN end nodes can follow 3 classes (with an increasing energy consumption level):
  - **Class A**: downlink windows are allowed only after an uplink (generally after 1 or 2 seconds)
  - **Class B**: downlinks are allowed in periodic windows → The gateway should transmit synchronization beacons to correctly manage the communication
  - **Class C**: it's the «continuous listening» mode. The downlink window is always open after the first uplink, except when the device is transmitting
- The more common classes are A and C

LoRaWAN recap – Join methods

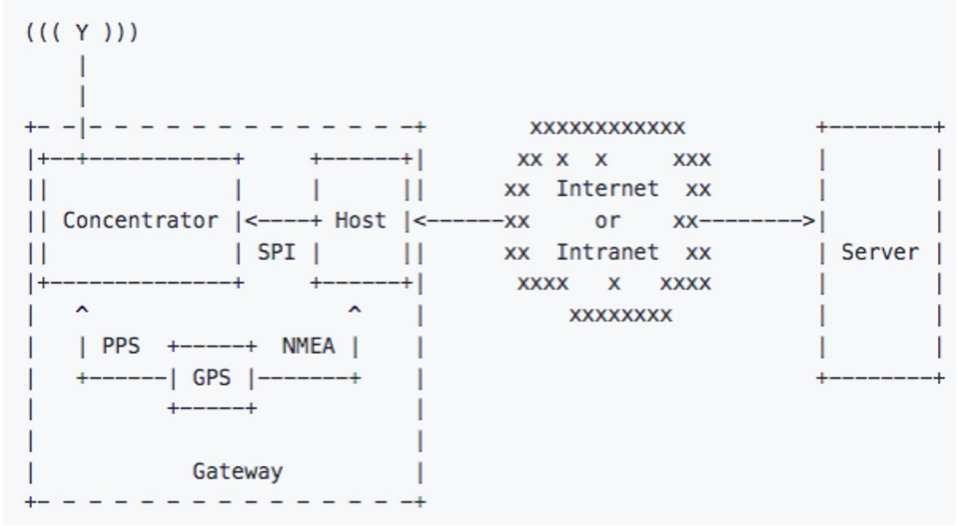
- Before starting sending uplinks, the end nodes have to join the LoRaWAN network
- Two possibilities are available:
  - **ABP**: Activation By Personalization (static)
    - DevAddr (network address)
    - NwkSKey (network session key)
    - AppSKey (application session key)
  - **OTAA**: Over-The-Air Activation (recommended)
    - DevEUI
    - AppEUI/JoinEUI
    - AppKey (application Key)
- In OTAA Session keys are generated dynamically at each join

LoRaWAN Protocol versions

- LoRaWAN version 1.0 (Jan 2015): is the first approved version of LoRaWAN
- LoRaWAN version 1.0.1 (Feb 2016): This minor revision clarified some points that were not clear in the previous version. Moreover, added some frequency plans, and other aspects
- LoRaWAN version 1.0.2 (July 2016): This minor revision added more MAC commands, fixed some errors in the previous version, and moved the section "Physical Layer" to a separate document
- LoRaWAN version 1.1 (Oct 2017): This major revision presented a new architecture
  - A new server called Join Server (JS) is introduced
  - The Network Server entity is split into three servers: Home Network Server (hNS), Serving Network Server (sNS), and forwarding Network Server (fNS)
  - This architecture enables roaming services between network operators
- LoRaWAN version 1.0.3 (Jul 2018): This minor revision related to v1.0.2 reused the same specifications for the class B devices mentioned in LoRaWAN version 1.1, added some MAC commands for class A devices, and deprecated some others
- LoRaWAN version 1.0.4 (Oct 2020): This minor revision related to v1.0.3 (clarifications and security mechanisms)

LoRaWAN Gateways

- Gateways are the bridge between devices and the Network Server
- Devices use low power networks (LoRa) to connect to the Gateway, while the Gateway uses high bandwidth networks like WiFi, Ethernet or Cellular to connect to the Network Server



Each LoRaWAN Gateway:

- Contains an hardware component called Concentrator that is the transmitter/receiver module designed to manage LoRa packets on multiple channels (e.g. Semtech SX1302 or Semtech SX1303)
- Is equipped with a Packet Forwarder, a software component that forwards packets received by the concentrator to the NS through an IP/UDP link, and sends packets that are received from the NS

LoRaWAN Gateways – Packet Forwarders

- The most common Packet Forwarders used in the LoRaWAN market are both developed by Semtech

Semtech's UDP Packet Forwarder:

- It is the first of its kind
- Simple and lightweight
- It uses UDP connections to transfer LoRaWAN packets from the gateway to the network server and vice versa
- widely used: many manufacturers preinstall it on their gateways
- No data encryption (between GW and NS)
- UDP can be unreliable because there are no data reception guarantees or confirmation from the UDP protocol

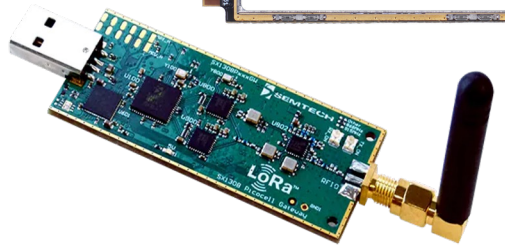
Semtech's LoRa Basic Station:

- Released in early 2019
- It uses WebSockets to transfer packets to the Network Server and vice versa
- Support end to end data encryption (TLS)
- Allows the connection to a Configuration and Update Server (CUPS) to manage and update gateways configurations
- CUPS allow users to access remotely to the gateway
- Better choice for large deployment

LoRaWAN Gateways 2/2

You can usually find two kind of gateways in the market:





- Gateways with a **minimal firmware**, making them low-cost and easy to integrate with other boards, running only the Packet Forwarding software



- Gateways with a complete **operating system**, where the packet forwarding software run as a background program. This gives more control to the gateway administrators to manage and add functionalities to the gateway and to install their own software

LoRaWAN Network and Application Servers

- Network Server and Application Server are often deployed on the same machine
- They can be deployed on the Cloud or managed in a «local» machine (e.g.: your PC or a Raspberry Pi)
- Some open-source options are nowadays available to manage data from a LoRaWAN network:
  - The Things Network (TTN)
  - ChirpStack
- After the Application Server, data can be provided to external entities (applications) with different options: HTTP REST APIs, MQTT, Web Sockets and other…

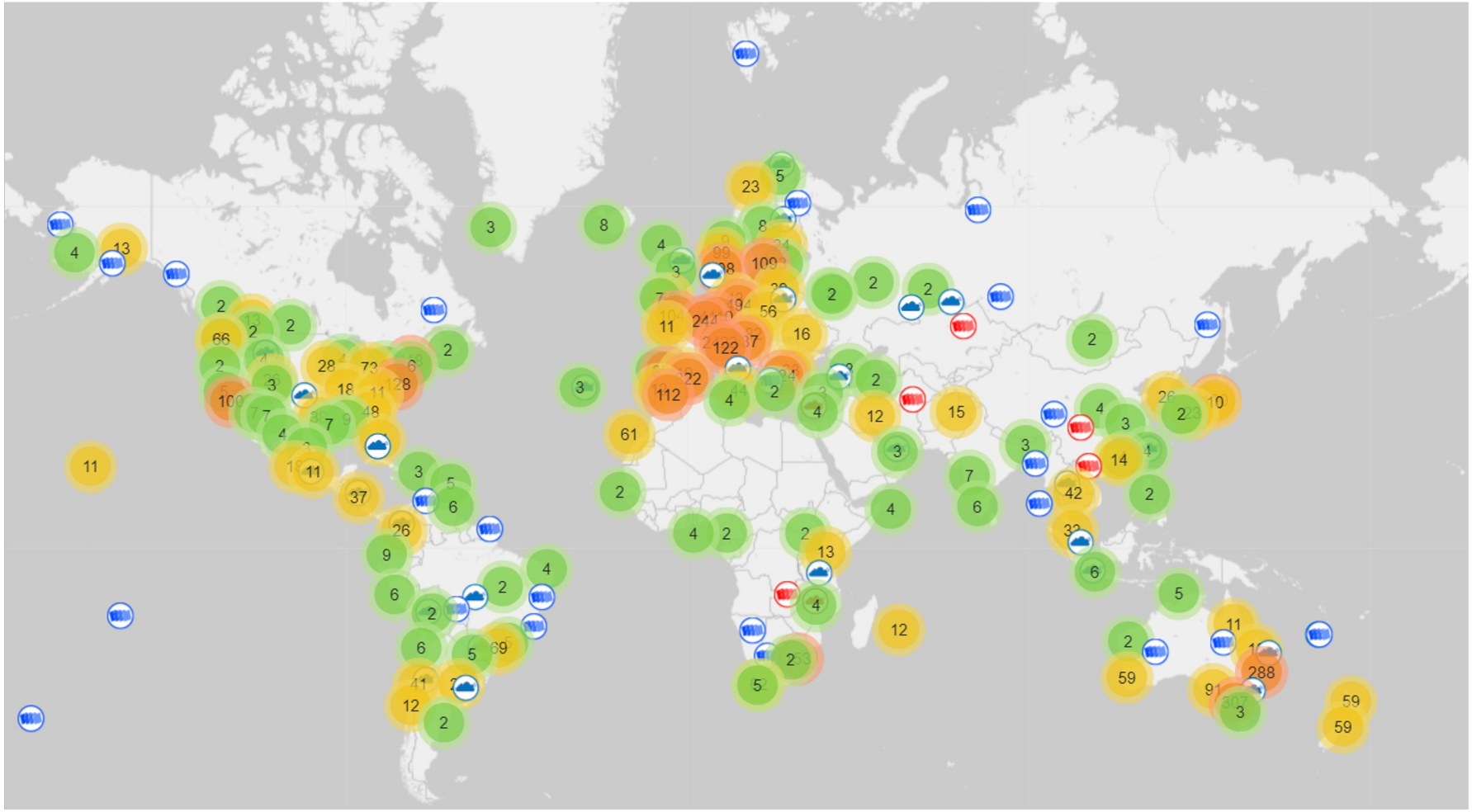
The Things Network (TTN)



- Is a global collaborative Internet of Things (IoT) ecosystem that creates networks and solutions using the LoRaWAN protocol
- Runs on The Things Stack Community Edition, which is a open and decentralized LoRaWAN Network Server implementation distributed over the globe
- TTN provides developers and students with a set of open tools to get started testing devices, and get familiar with LoRaWAN network components and applications
- It is based on The Things Stack which is an Enterprise LoRaWAN Network Server solution which can be used by companies as a commercial product

The Things Network (TTN)

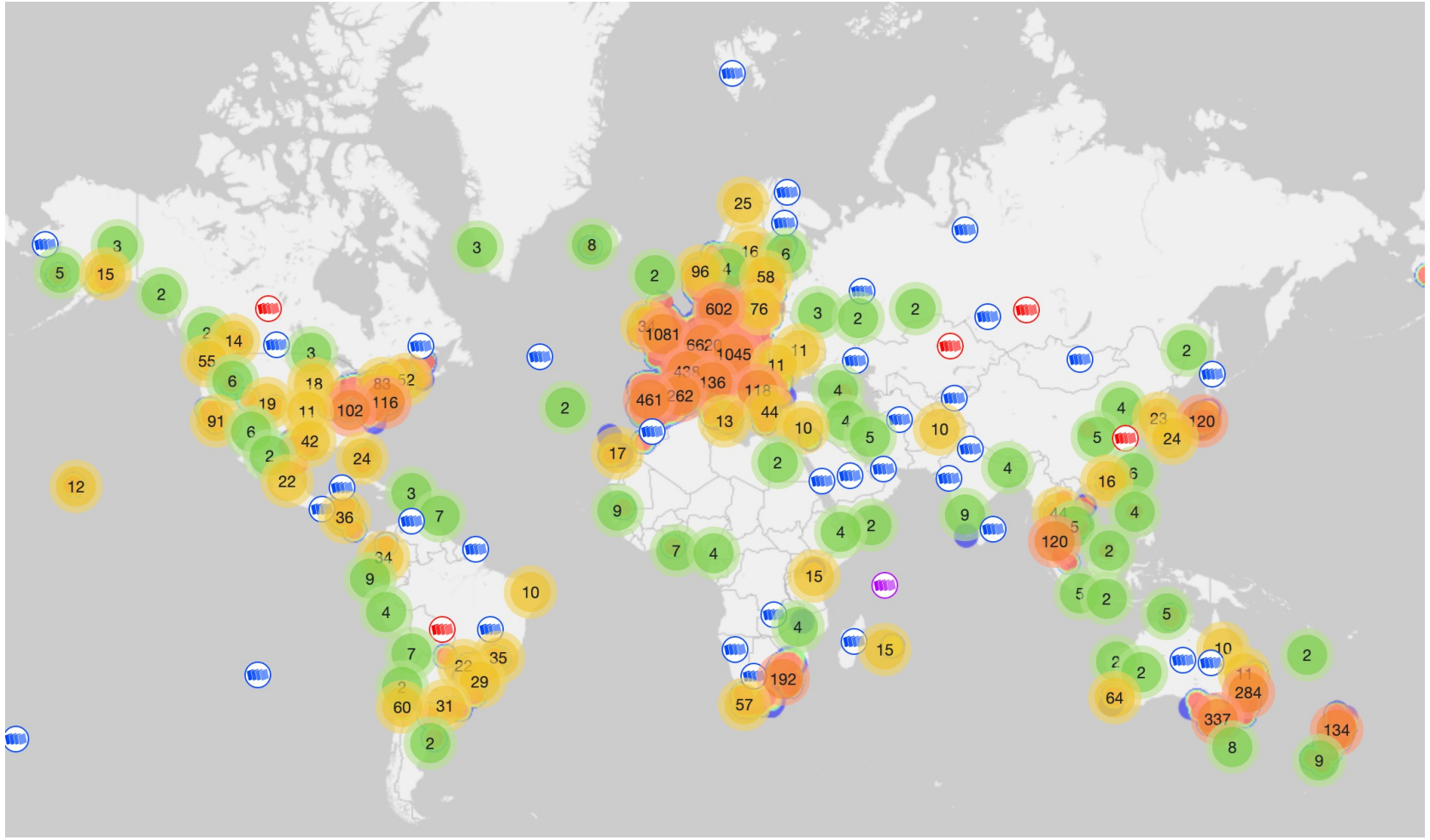
Gateways deployed on TTN (2024)



THE THINGS NETWORK

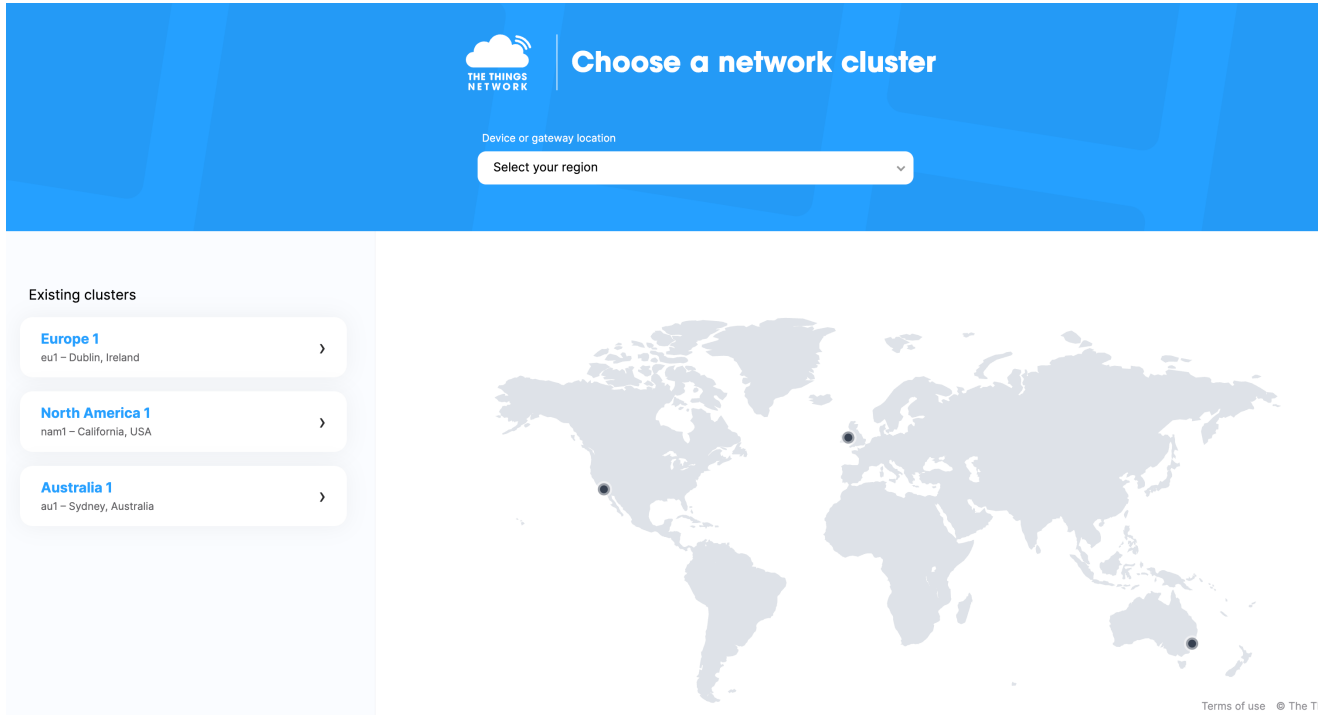
The Things Network (TTN)

Gateways deployed on TTN (2025)



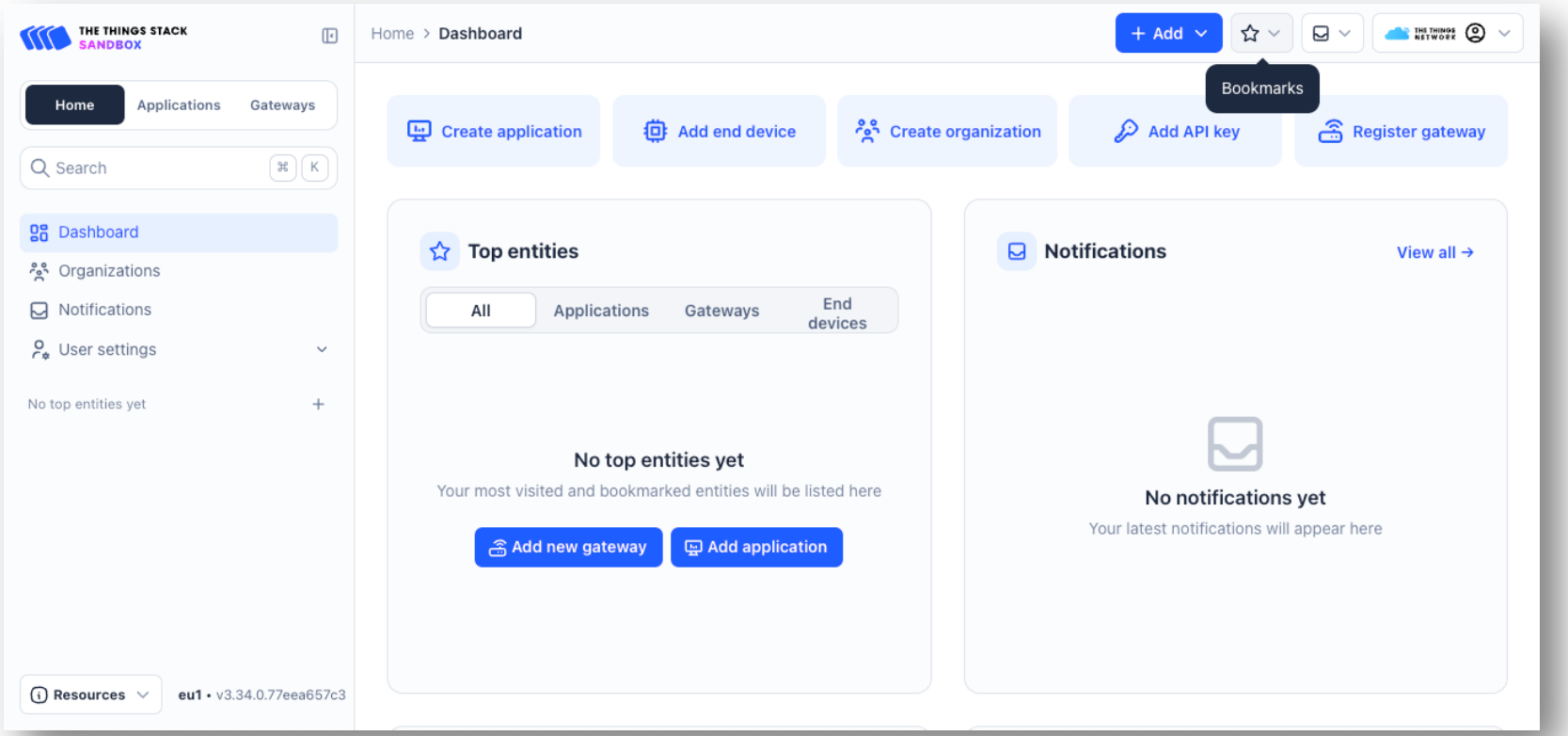
The Things Network (TTN)

- You can create a free account in the Things Stack Community Edition website
- It's a multi-cluster deployment: this means that with your account you can connect your gateways to a cluster, and route all your IoT traffic in that cluster



- This limitation is done to significantly reduce latency: you are assigned to a cluster, so your traffic would not have to cross half the planet
- Currently 3 clusters are available, for different geographic areas

The Things Network: Web console



Each account can access to the TTN Console which is the management application of the Things Stack for LoRaWAN

- It is a web based application accessible from the TTN website
- It is used to register applications, end devices or gateways
- It allows to monitor network traffic, or configure network related options

The Things Network: Gateway entity (1/2)

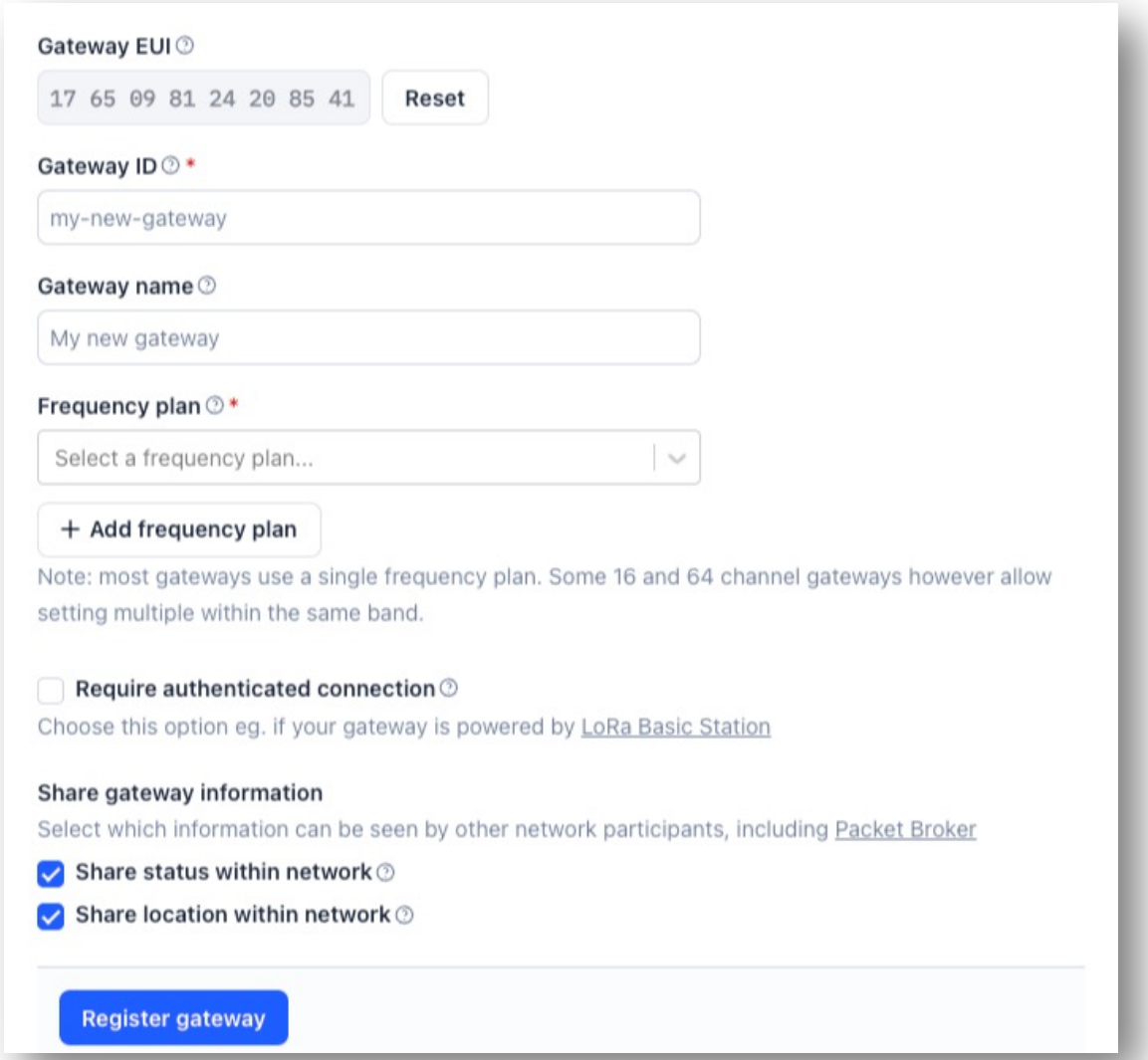
Relevant parameters:

- ID: a unique textual identifier for the gateway

  e.g.: `gw-iotlab-campus-unipr`

- EUI: a unique 64 bits identifier for the gw (provided by manufacturer)

  e.g.: `1765098124208541`

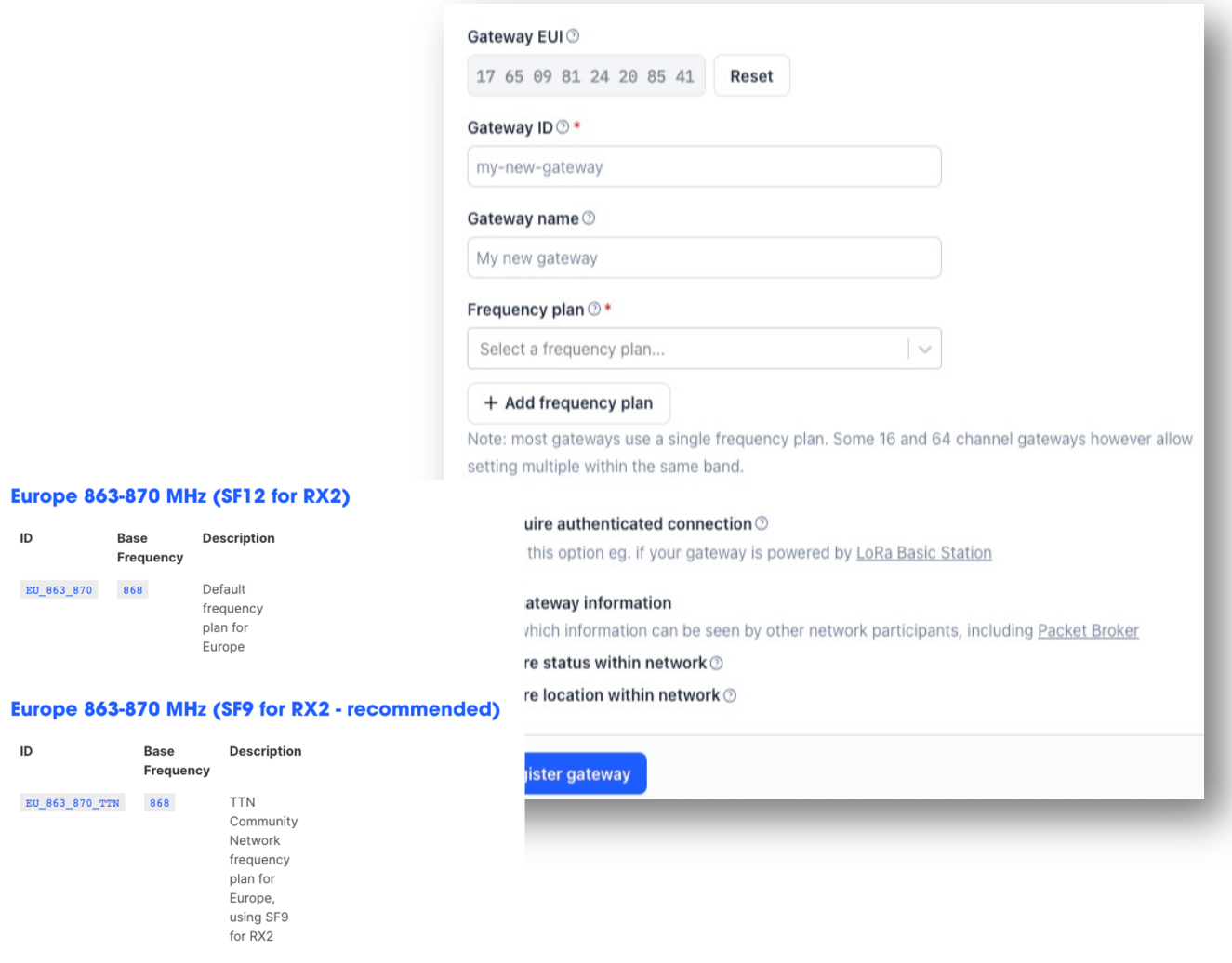


The Things Network: Gateway entity (2/2)

- Frequency plan: defines data rates and channels which comply with the LoRaWAN Regional Parameters for a geographic area

  e.g.: `EU_863_870_TTN`

- Frequency plans are defined by the LoRa Alliance Technical Committee and are published as the LoRaWAN Regional Parameters technical document

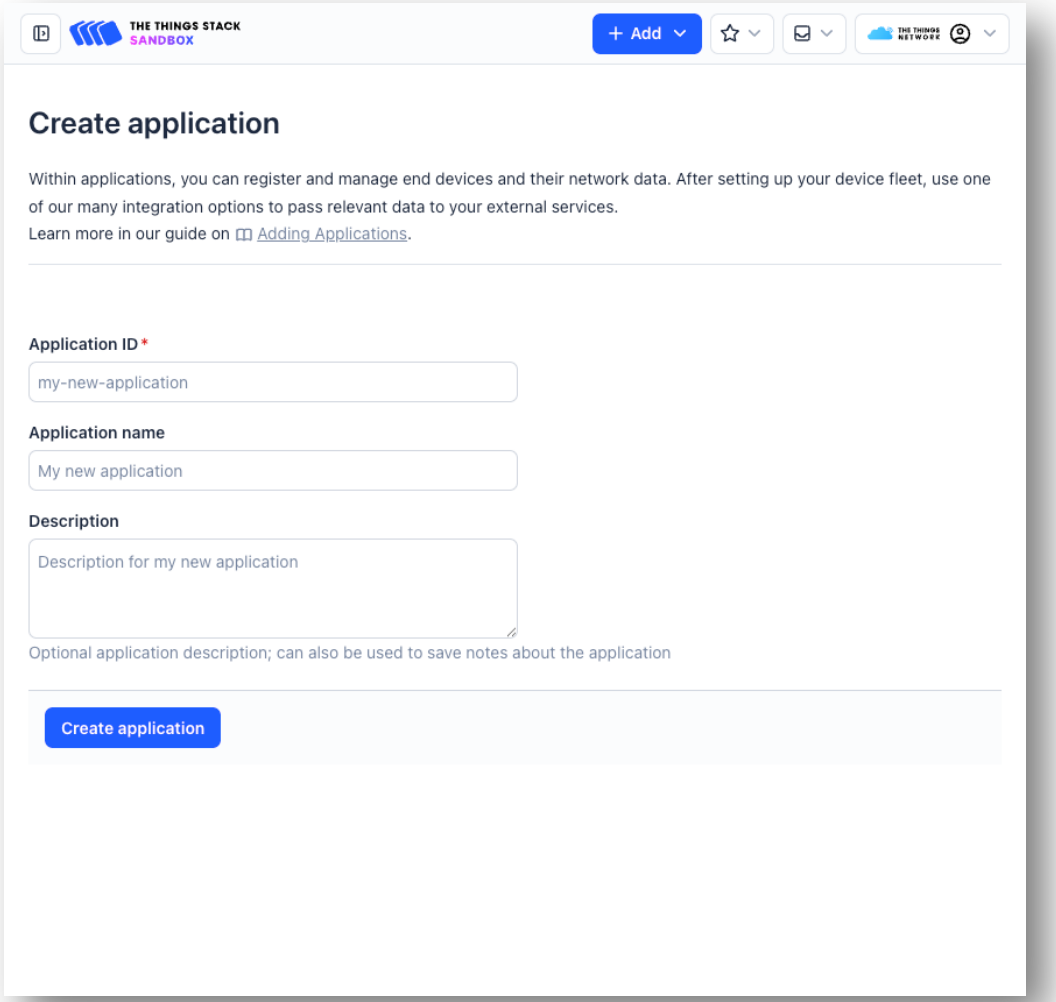


The Things Network: Application entity

- An Application in TTN environment is used to represent a collection of LoRaWAN devices with the same purpose / of the same type

  e.g.: LoRaWAN weather stations collecting data at different locations
- Application also allows to manage integrations for data
- Relevant Parameters:
  - ID: a unique textual identifier for the application

    e.g.: `smart-unipr-campus`



The Things Network: End Device Entity (1/2)

- LoRaWAN end nodes must be registered under an application
- Relevant parameters:
  - Frequency plan: should be the same of the registered gw!

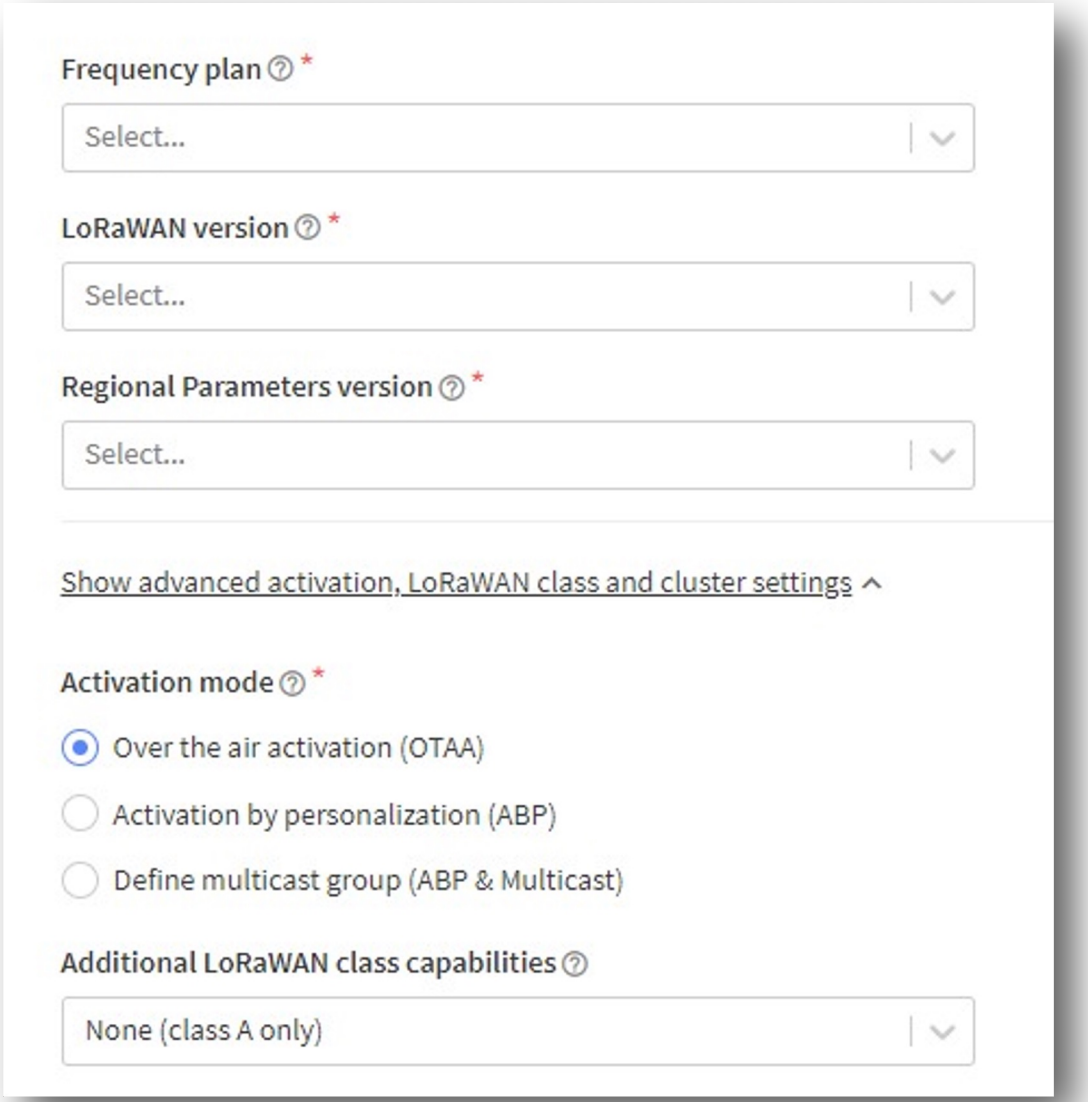
    e.g.: `EU_863_870_TTN`

  - LoRaWAN protocol version: the LoRa Alliance LoRaWAN specification your device conforms to, which defines which MAC features it supports. It is provided by manufacturer

    e.g.: `1.0.2 or 1.0.4`

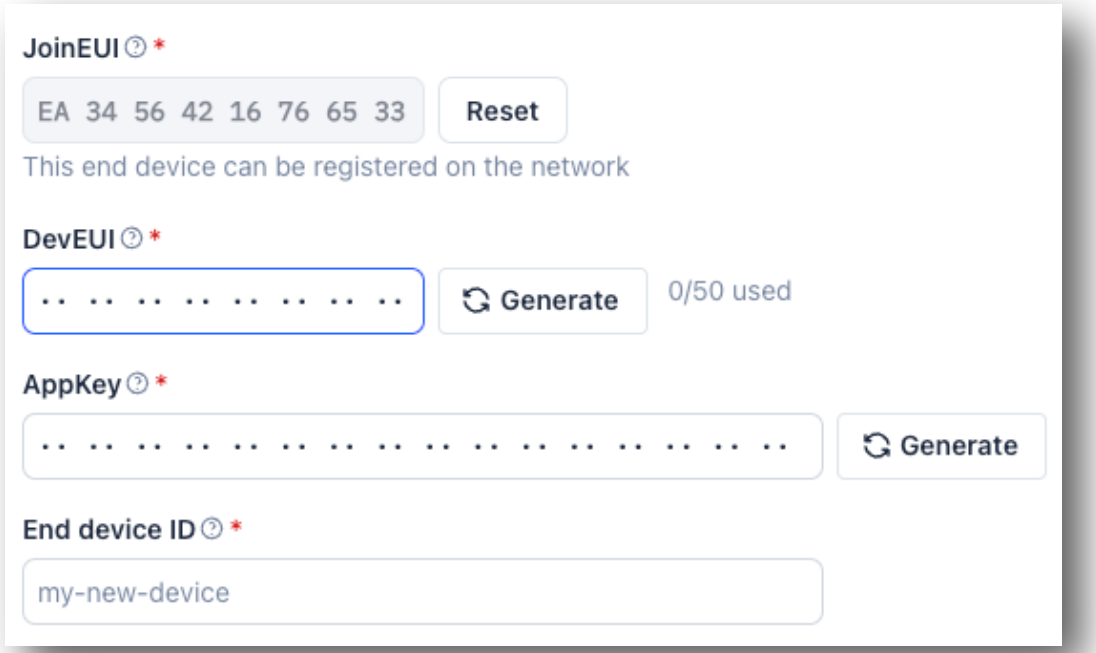
  - Class capabilities: class A, B, or C

    e.g.: `A`

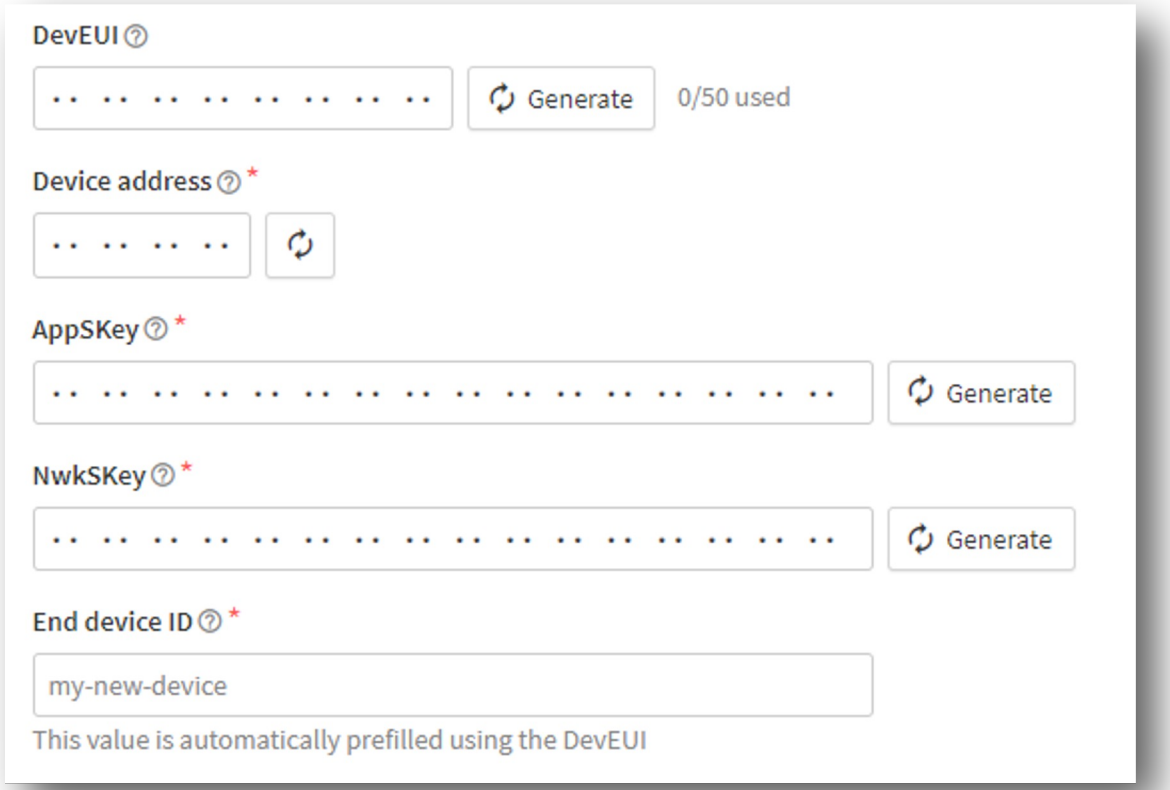


The Things Network: End Device Entity (1/2)

- DevEUI: a 64 bit unique identifier for the device (is provided by the manufacturer and should not be changed)
- End device ID: textual identifier



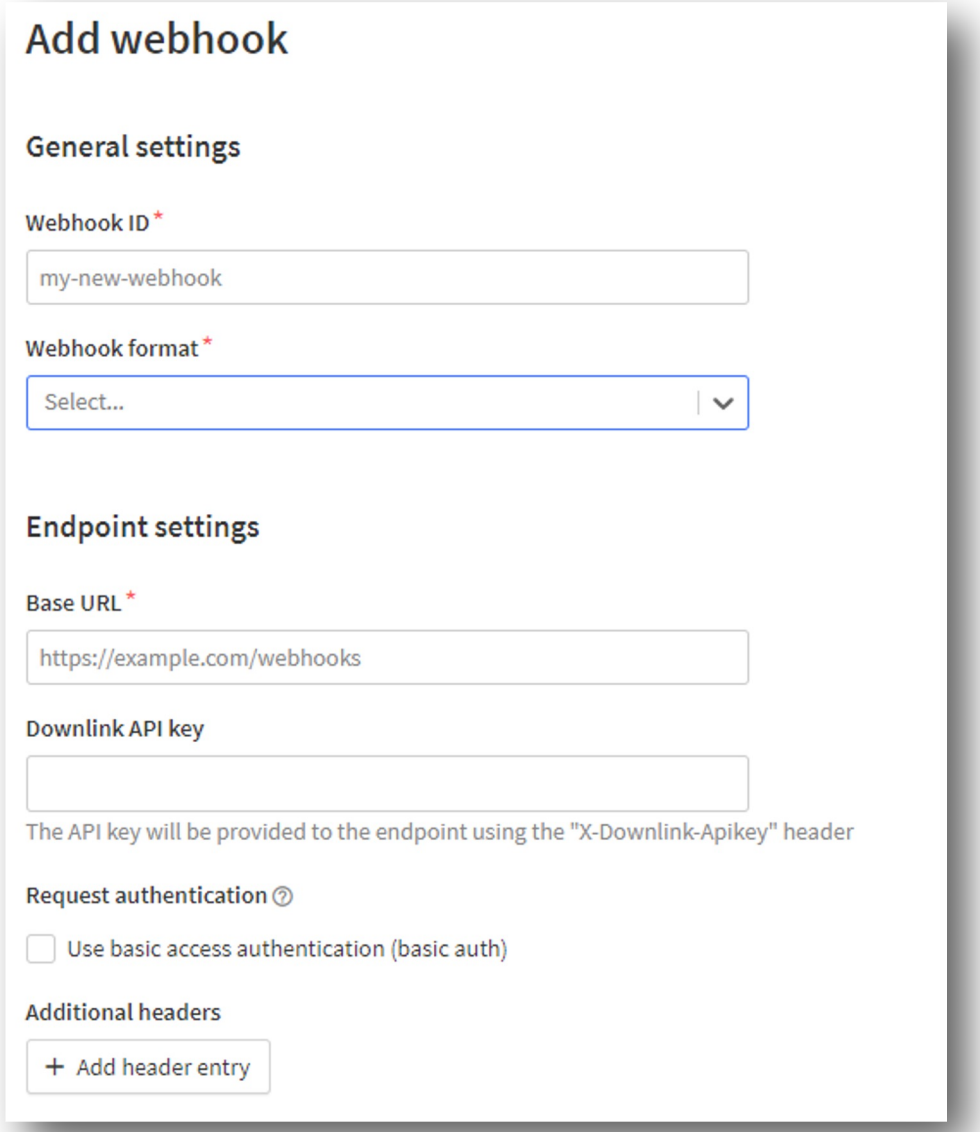
OTAA Activation parameters



ABP Activation parameters

The Things Network: Integrations

- Integrations connect devices to the external world (e.g.: custom external applications)
- They allow to trigger events whenever a device communicates (join, uplink, downlink)
- The Application server manages integrations using Webhooks (namely HTTP POST calls), MQTT, or other
- Supported data format: JSON or ProtoBuf

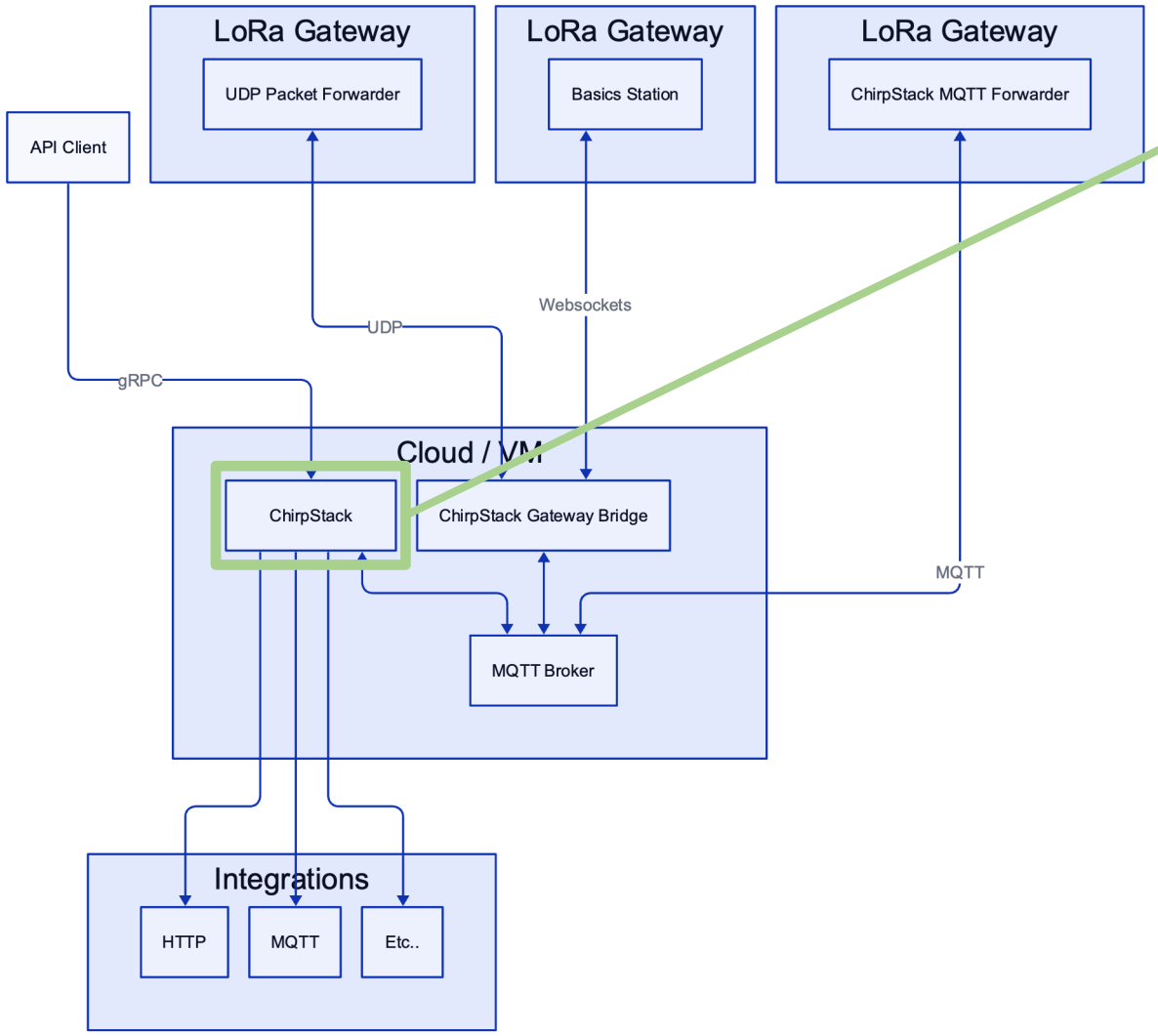


ChirpStack



- ChirpStack is an open-source LoRaWAN Network Server stack providing a set of completely open-source software components to build LoRaWAN networks
- Ready-to-use solution that can be installed on private servers or personal PC
- Includes a user-friendly web-interface for LoRaWAN network devices and gateways management and for integration
- All components are licensed under the MIT license and can be used for commercial purposes

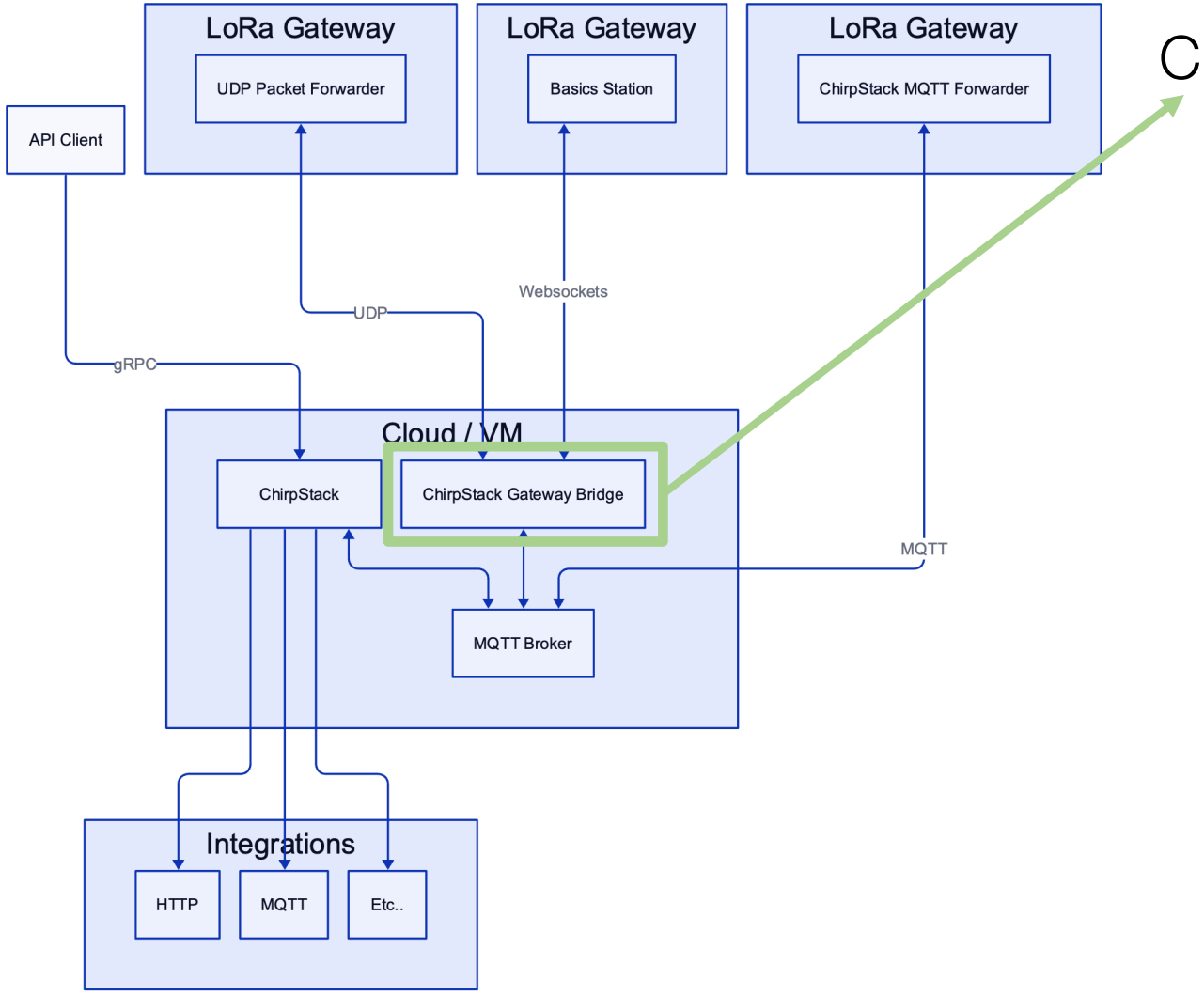
ChirpStack Architecture Main Components



ChirpStack:

- The LoRaWAN Network Server + Application Server implementation
- It provides a web-interface for the gateways, the devices and the integrations management
- Provides API that can be used to integrate or to extend ChirpStack

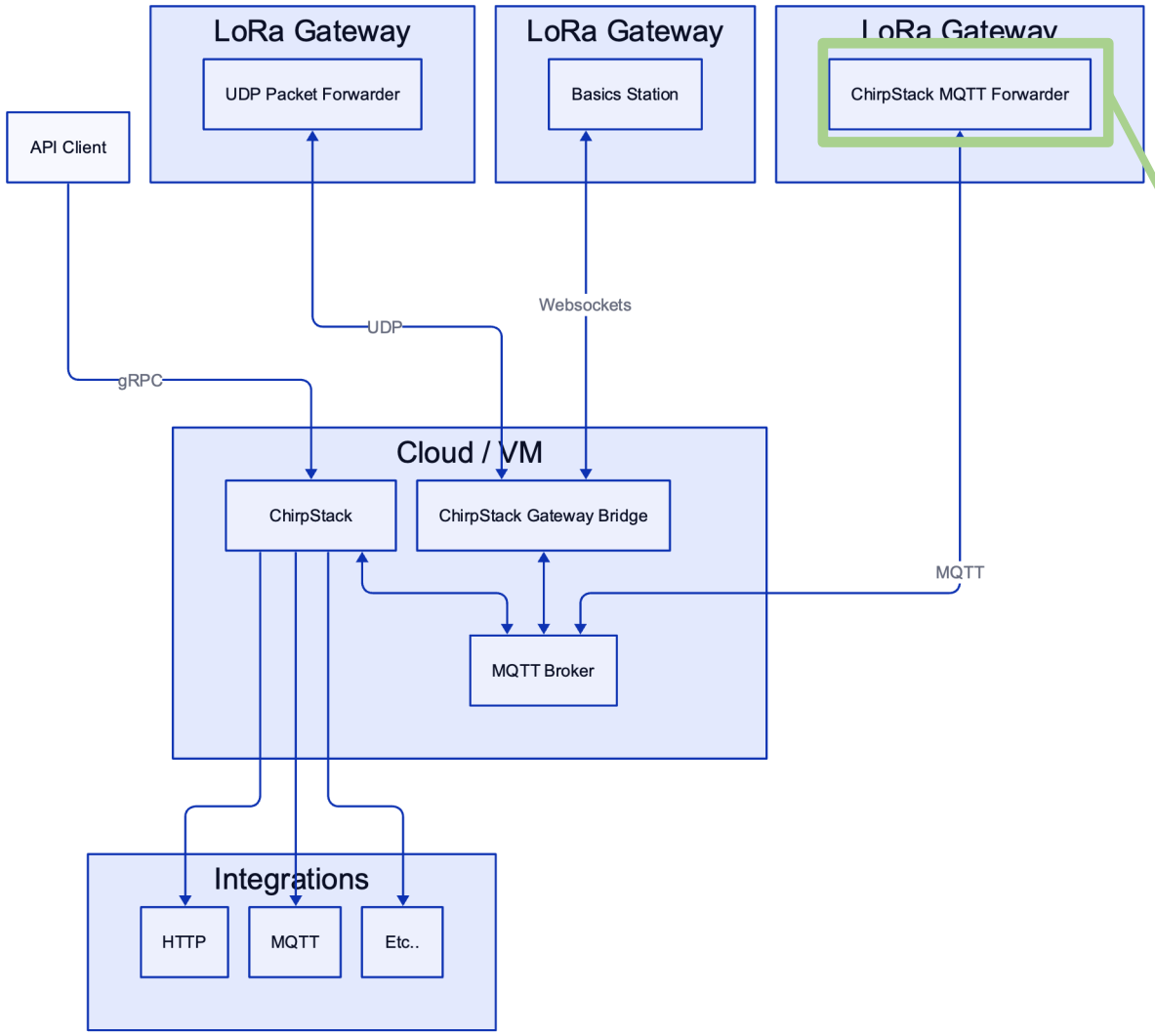
ChirpStack Architecture Main Components



ChirpStack Gateway Bridge:

- An open source bridge that handles the communication with the LoRaWAN gateways and the Network server
- It converts messages from the Packet Forwarder format (used in Semtech UDP Packet Forwarder or Semtech Basic Station) into the ChirpStack data format (MQTT)
- It can be installed on the server or on the gateways

ChirpStack Architecture Main Components



Other relevant components:

- ChirpStack Concentratord: the first ChirpStack implementation of the Packet Forwarder software (simpler)
- ChirpStack MQTT Forwarder: the open source ChirpStack implementation of the Packet Forwarder software (to be installed on gateways) based on the MQTT protocol

ChirpStack Installation (1/2)

- ChirpStack can be installed on Debian or Ubuntu distributions, but also on Raspberry Pi boards
- Before starting ChirpStack installation you should install some dependencies:
  - **MQTT broker**: a publish/subscribe protocol that allows users to publish information under topics that others can subscribe to. A popular implementation of the MQTT protocol is Mosquitto
  - **Redis**: an in-memory database used to store relatively transient data
  - **PostgreSQL**: the long-term storage database used by the open source packages

ChirpStack Installation (2/2)

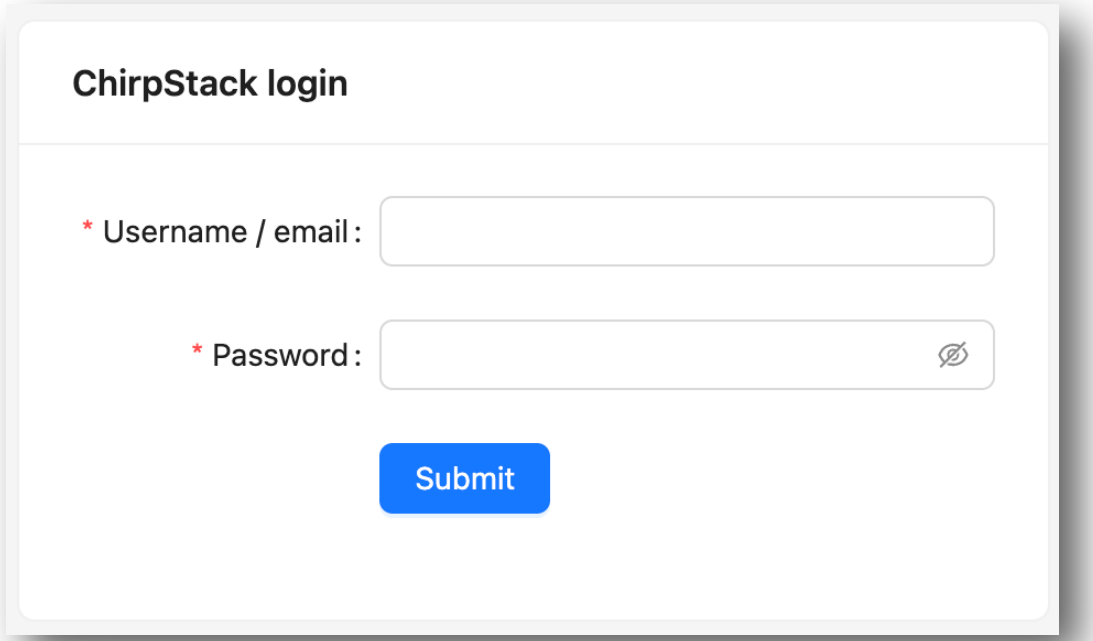
- You can install the ChirpStack Gateway Bridge on the Server in order to manage gateways not mounting it
- The ChirpStack NS supports all regions that are defined by the LoRaWAN Regional Parameters specification.

  → It should be configured for a specific region (e.g.: EU-868) through a JSON configuration file
- After installing all components you can access the web interface as admin at:
- `http://<my-server-ip>:8080`

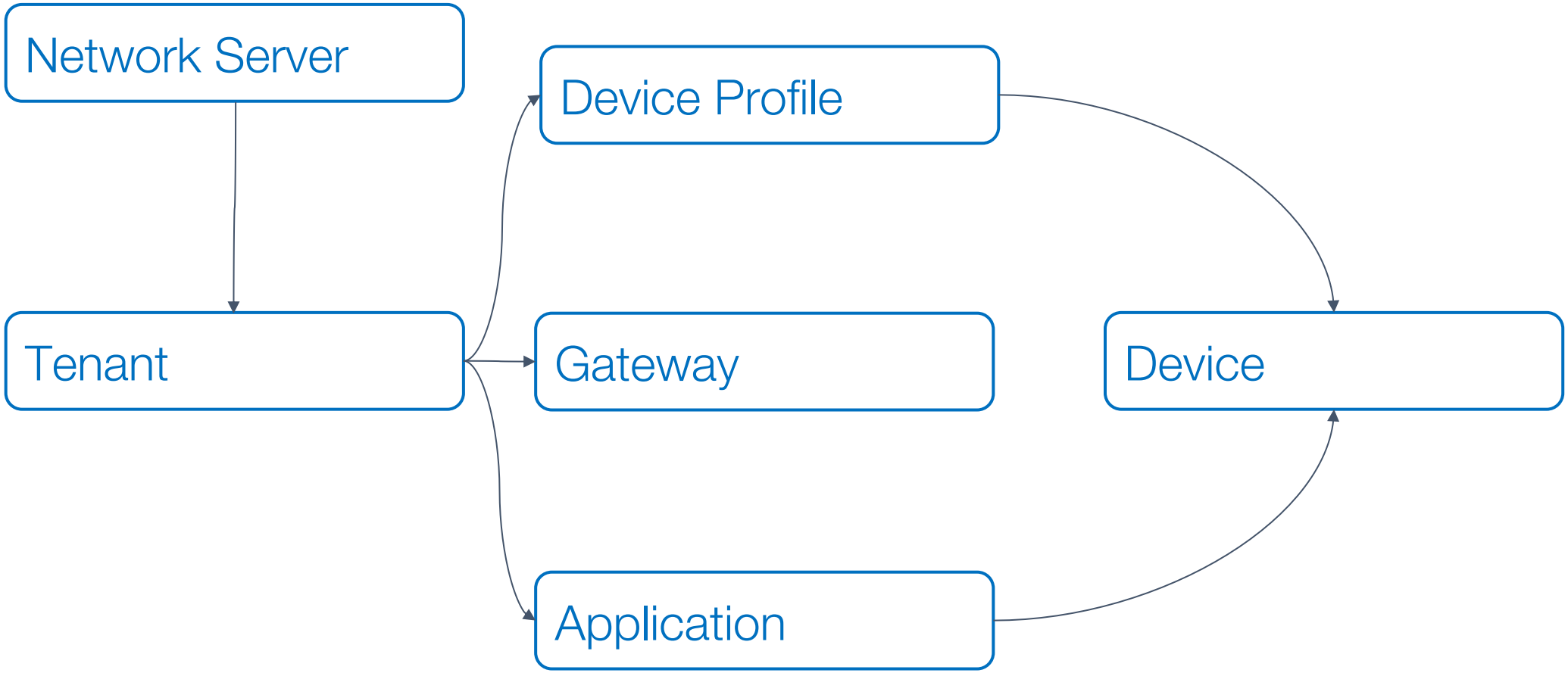
```
sudo apt install chirpstack-gateway-bridge
…

sudo apt install chirpstack
…

# start chirpstack
sudo systemctl start chirpstack
```



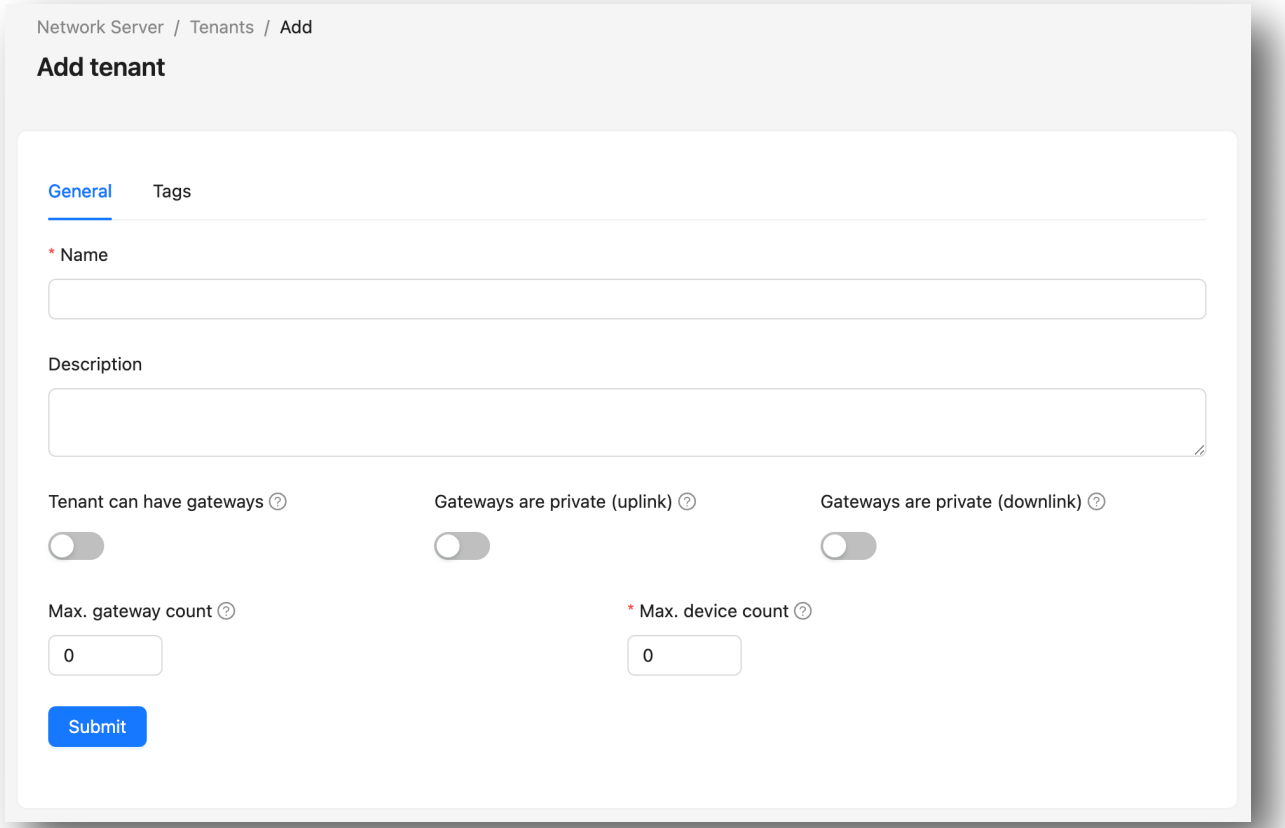
ChirpStack Entities Summary



ChirpStack Entities: Tenants

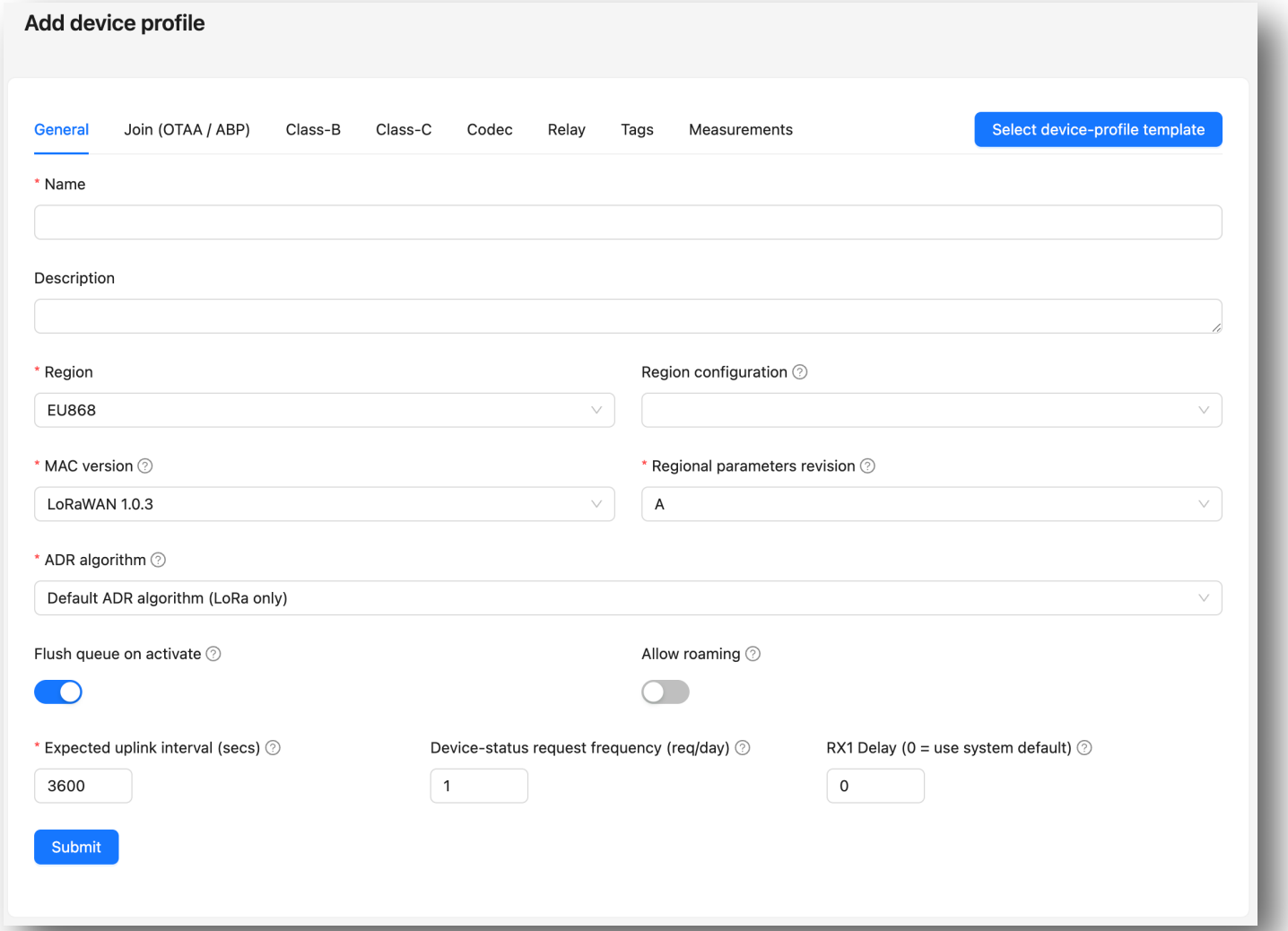
- Tenants can be used to let organizations or teams manage their own applications and optionally their own gateways

  → The admin account has visibility of all tenants
- A tenant can have:
  - Device Profiles
  - Gateways (when allowed)
  - Applications
  - Users



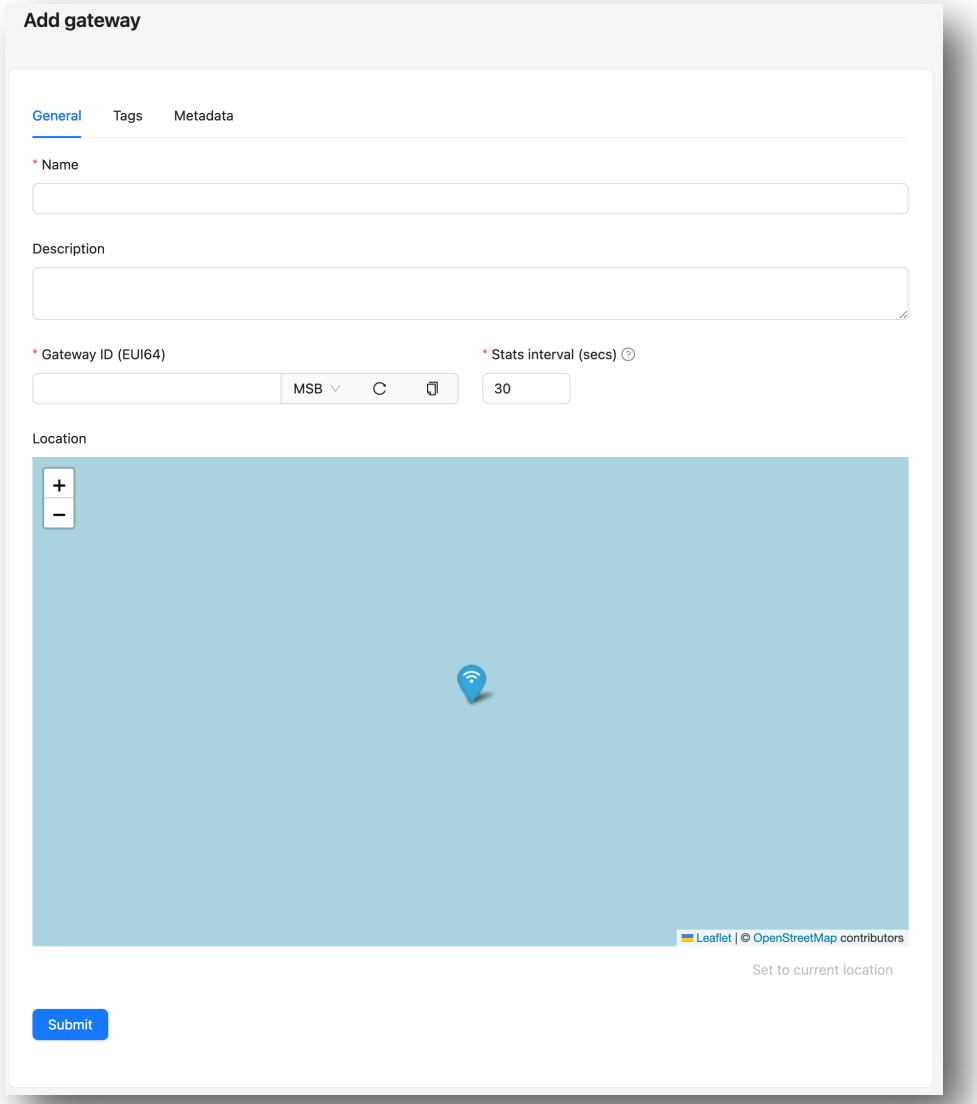
ChirpStack Entities: Device Profiles

- Device Profile: defines the device capabilities and parameters that are needed by the NS for setting the LoRaWAN radio access service
- It allows to group end nodes with the same behavior based on:
  - LoRaWAN protocol version and Regional parameters
  - Activation mode
  - Class
  - Codec
- These information elements shall be provided by the end-device manufacturer
- When creating a Device Profile, you have to specify the Tenant it belongs



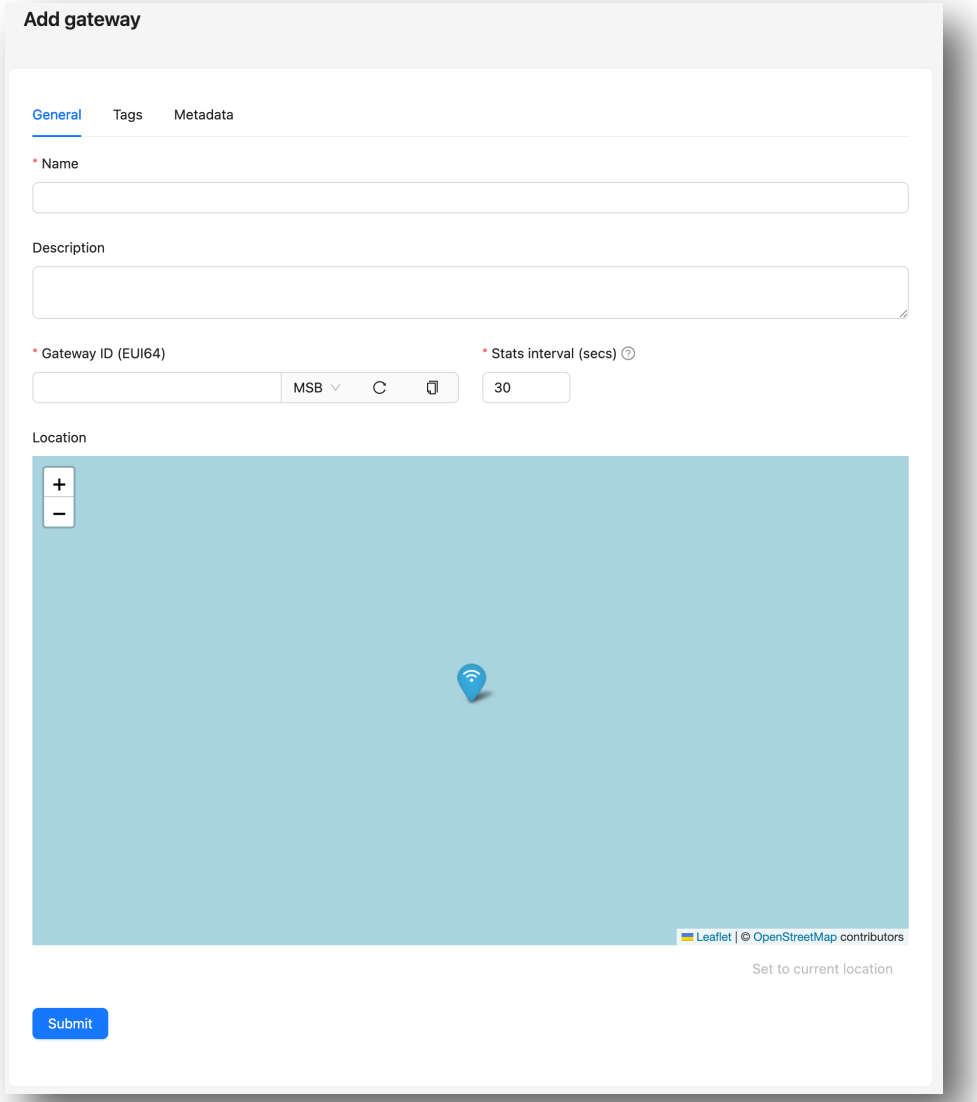
ChirpStack Entities: Gateway (1/2)

- Gateway: to register the Tenant gateways (if enabled)
- Similar to the TTN Gateway entity
- If a gateway belongs to a tenant does not mean that the usage of the gateway is limited to the tenant. Every device in the whole network will be able to communicate using the gateway
- The tenant will be responsible for managing the gateway details (e.g. name, location) and will be able to see its statistics



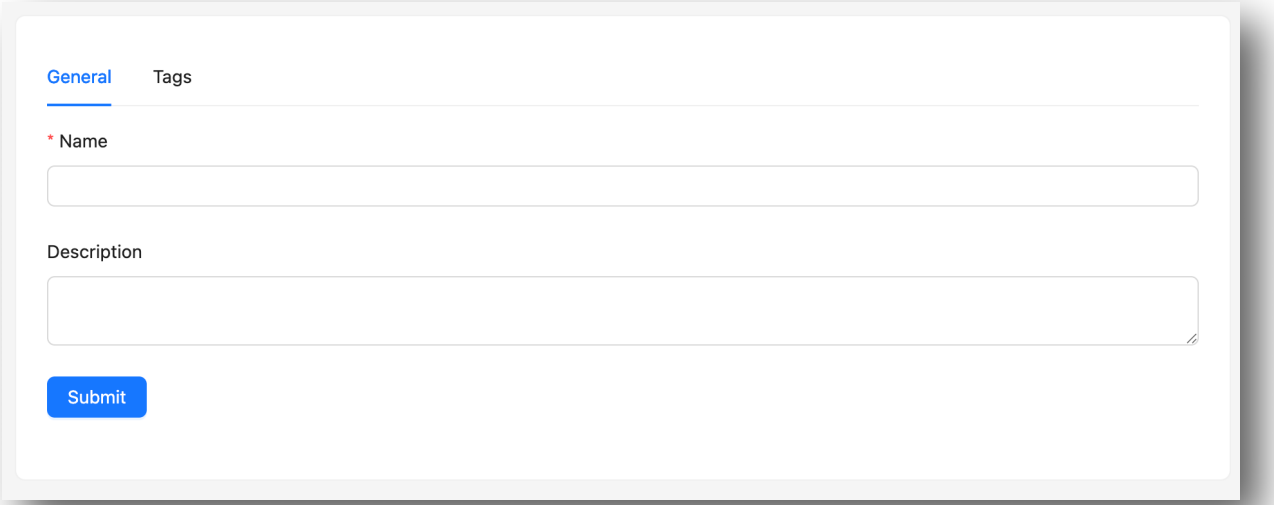
ChirpStack Entities: Gateway (1/2)

- Relevant parameters:
  - name: a unique textual identifier for the gw
  - ID: a hex encoded unique 64 bit identifier for the gw
    →**e.g.:** c64ae525e519bb9d
  - Geographical location



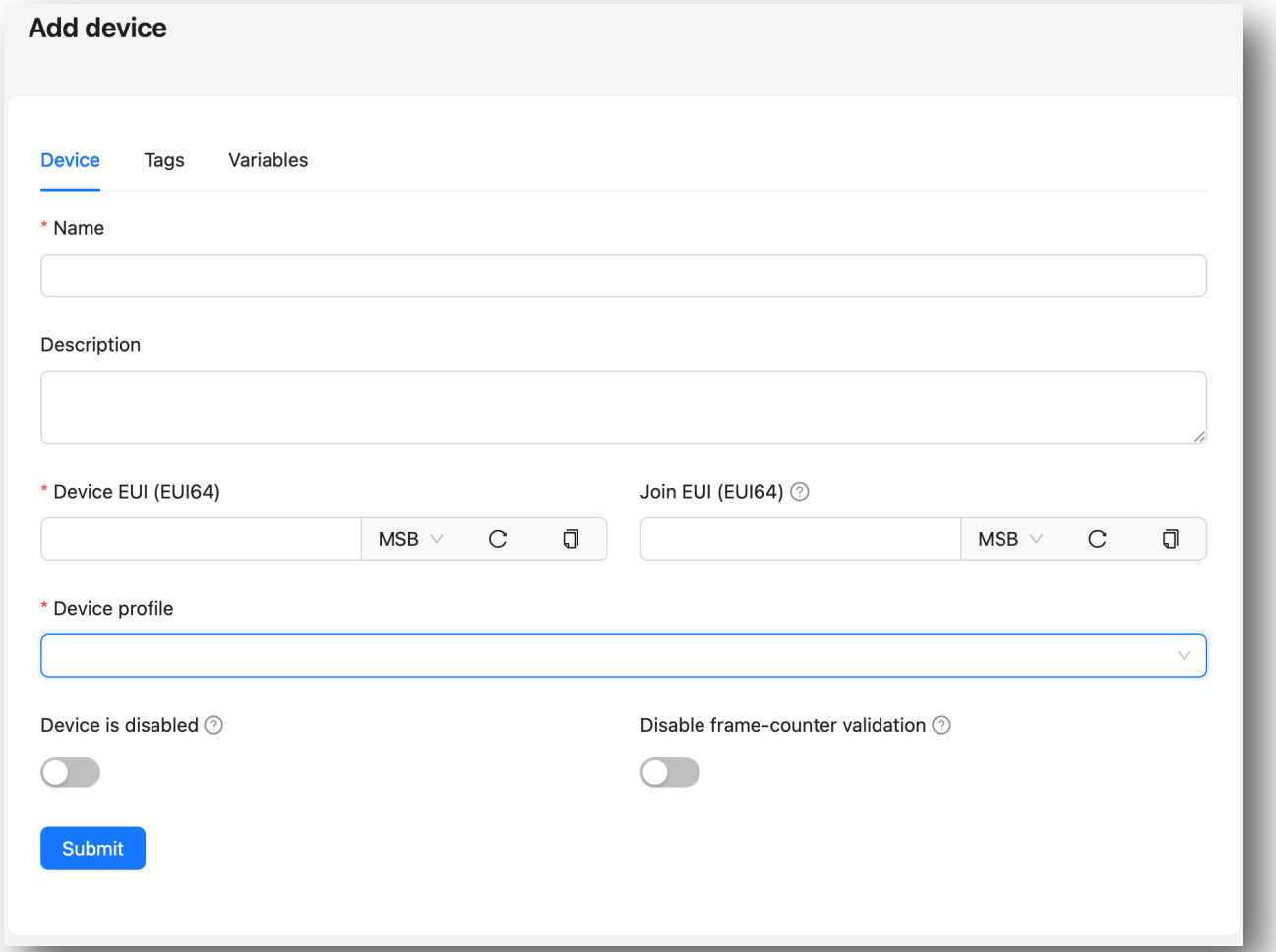
ChirpStack Entities: Application

- Application: is a collection of devices with the same purpose / type
- Similar to the TTN Application entity
- When creating an Application, you need to select the Tenant it belongs
- Multiple devices can be added to an application
- Integrations are intended at Application layer



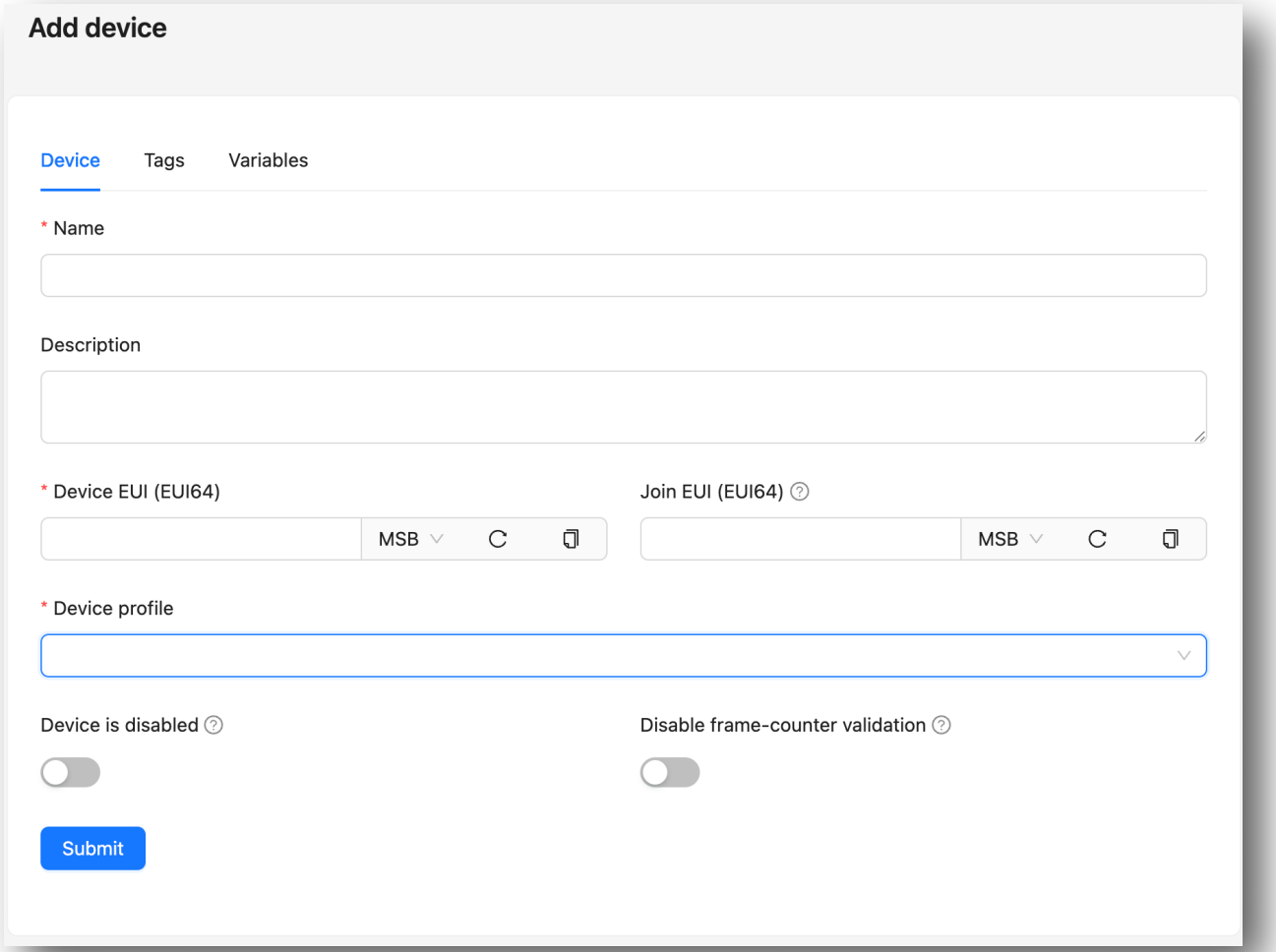
ChirpStack Entities: Device

- Devices: is the end-device connecting to, and communicating over the LoRaWAN network
- Device EUI: a hex encoded 64 bit unique identifier for the device
- When creating a Device, you have to select the application and the Device Profile, where are defined the join activation mode and other parameters



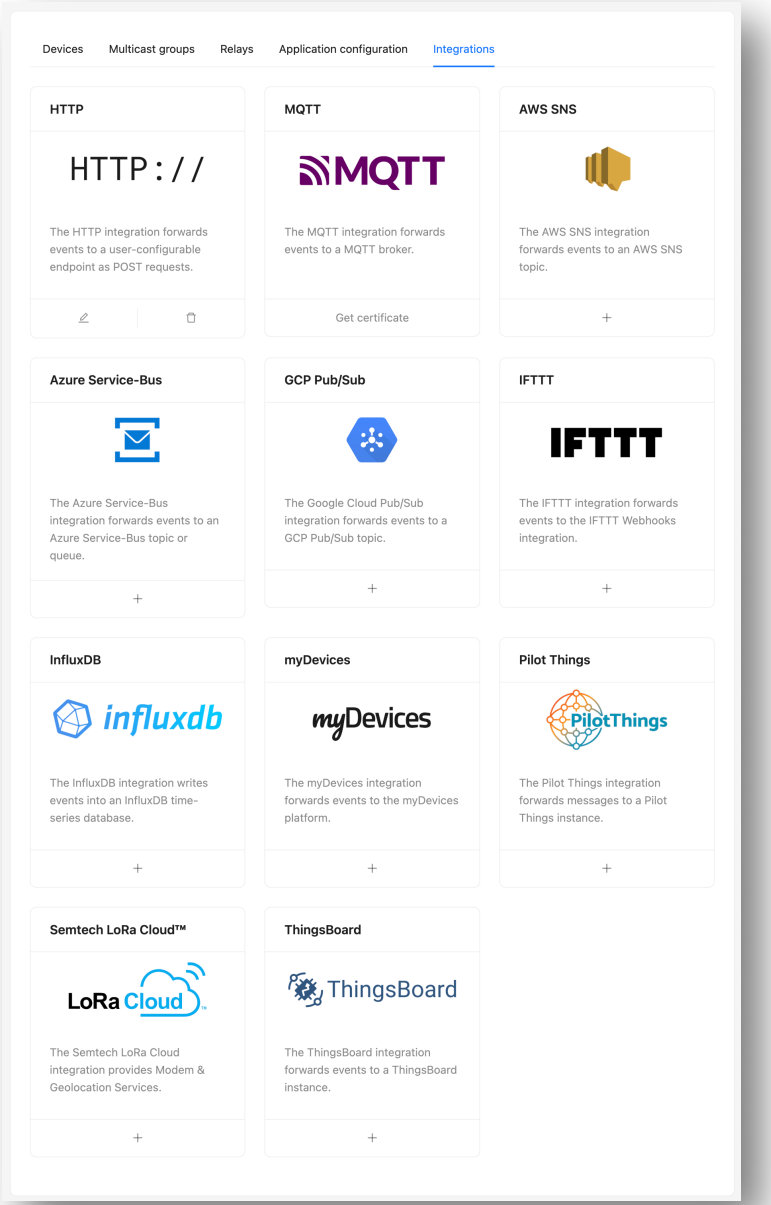
ChirpStack Entities: Device

- Device Activation
- After creating a device, you can manage the activation parameters
- An “activation” tab will appear with different fields depending on the device profile
- OTAA devices:
  - device EUI
  - application-key
  - appEUI (or JoinEUI)
- ABP devices
  - device address
  - network session key
  - application session key



ChirpStack – Integrations

- Like in TTN, Integrations connect devices to the external world custom applications
- They allow to trigger events whenever a device communicates (join, uplink, downlink)
- The Application server manages integrations using HTTP POST calls or MQTT
- ChirpStack also allow the use of HTTP APIs



TTN or ChirpStack?

Use TTN if:

- You simply want to test your devices
- You don’t have a gateway (maybe someone near you has registered his gw)
- You don’t have a host to use for NS installations
- If you want a global coverage

Use ChirpStack if:

- You need full control of network components
- If you want a particular behavior for gateways
- You want host NS and AS on your machine or on your device
- You want to create a private network

Devices: some examples @ IoTLab

- Microchip LoRa Pictail daughter board
- LoRaWAN transceiver module based on the Microchip RN2483 LoRa modem
- Can work as class A and C device
- LoRaWAN v1.0.2 compliant
- Can be connected to a host computer through a standard USB interface
- Can be controlled sending high level textual commands using serial port (e.g. using Putty)



RN2483 commands (some examples)

- Commands syntax is composed by a keyword followed by parameters
- Available keywords are:
  - “sys”: for system level behavior actions (e.g. gathers status or hw information)
  - “radio”: for radio specific configurations (LoRa level)
  - “mac”: for LoRaWAN communication configuration and actions

```
sys get ver //Returns the information on fw version
> RN2483 1.0.5 Oct 31 2018 15:06:52

sys get hweui //Retuns the globally unique EUI for device
> 0004a30b00f56978
```

```
radio set sf sf7
//sets the spreading factor used during transmission
> ok
```

```
mac set class c //Set the class for the device
> ok

mac join otaa //Join the network
> ok
> join accepted
```

Devices: some commercial examples @ IoTLab



Milesight EM500-PP: pipe pressure sensor



Milesight EM500-smtc: soil moisture, temperature and electrical conductivity sensor



Milesight EM500-CO2: CO_2, barometric pressure, humidity, temperature sensors

- Class A end-devices
- Work as standalone battery-powered nodes
- Can be configured through NFC communication and a manufacturer provided software

Devices: some commercial examples @ IoTLab



Milesight UC511:
Solenoid valve controller



Mclimate LorRaWAN water Valve



Milesight WS523:
Smart Plug,

- Class A end-devices
- Work as standalone battery-powered nodes

Devices: some commercial examples @ IoTLab



Decentlab LoRaWAN temperature leaf sensor



Decentlab LoRaWAN stem dendrometer



Decentlab LoRaWAN fruit dendrometer

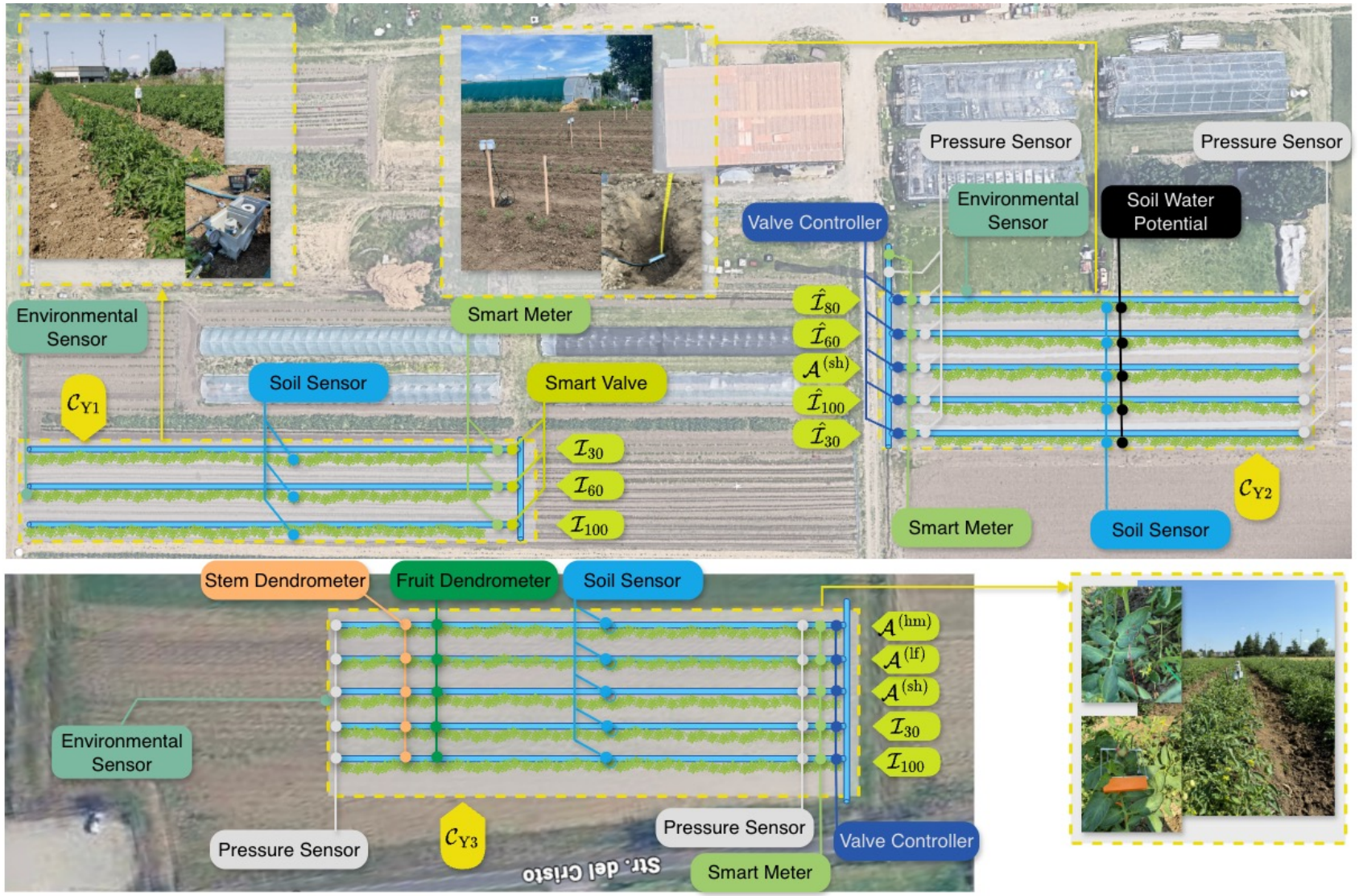
- Class A end-devices
- Work as standalone battery-powered nodes

A commercial Gateway

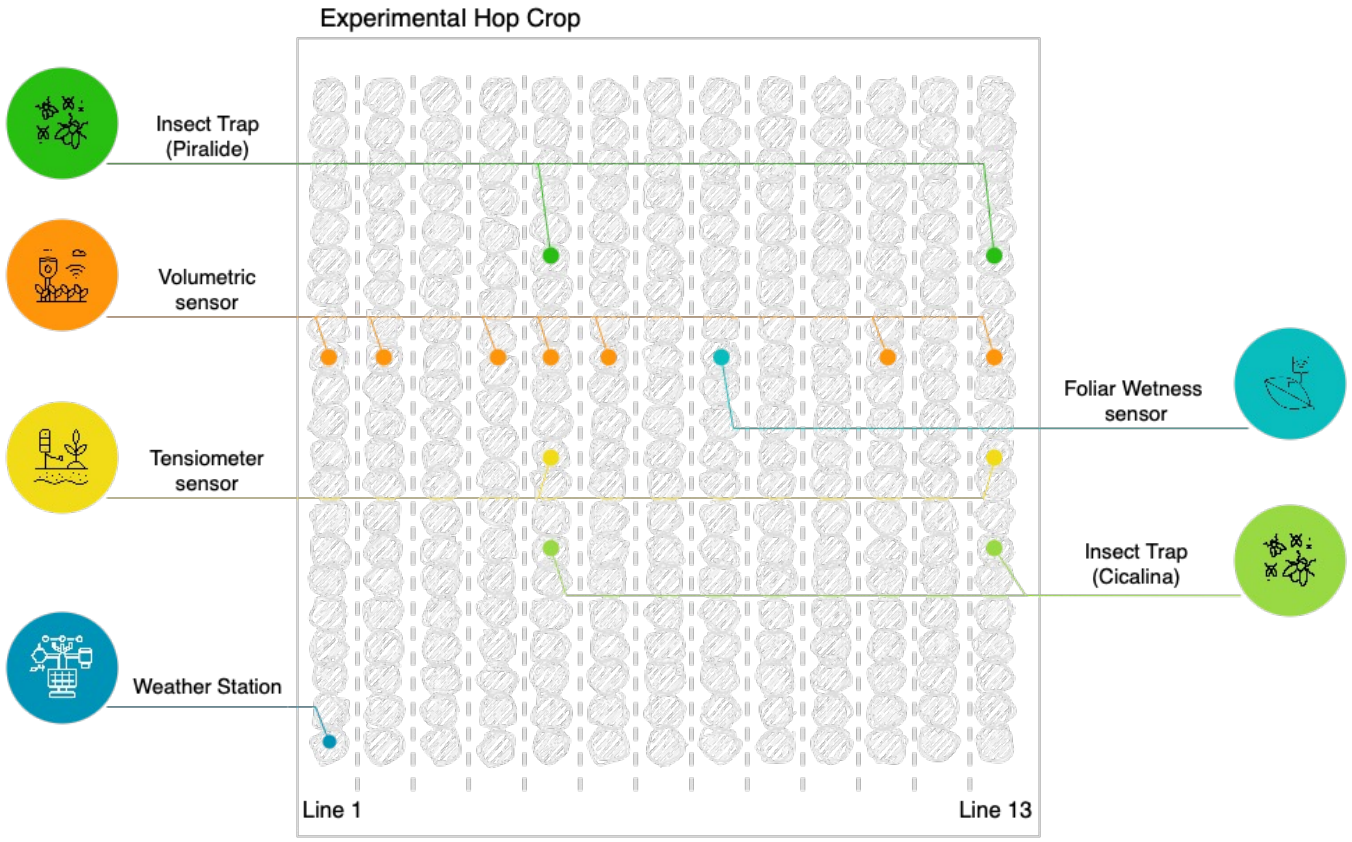
- Milesight UG67 Outdoor gateway
- Include an embedded Network Server, but it’s compatible with TTN and ChirpStack
- Compatible with the EU868 Frequency plan
- Support up to 2000 end devices
- Deployed @iotlab
- Connection to network server trough Ethernet



A smart agriculture project @ IoTLab



A smart agriculture project @ IoTLab





A smart agriculture project @ IoTLab



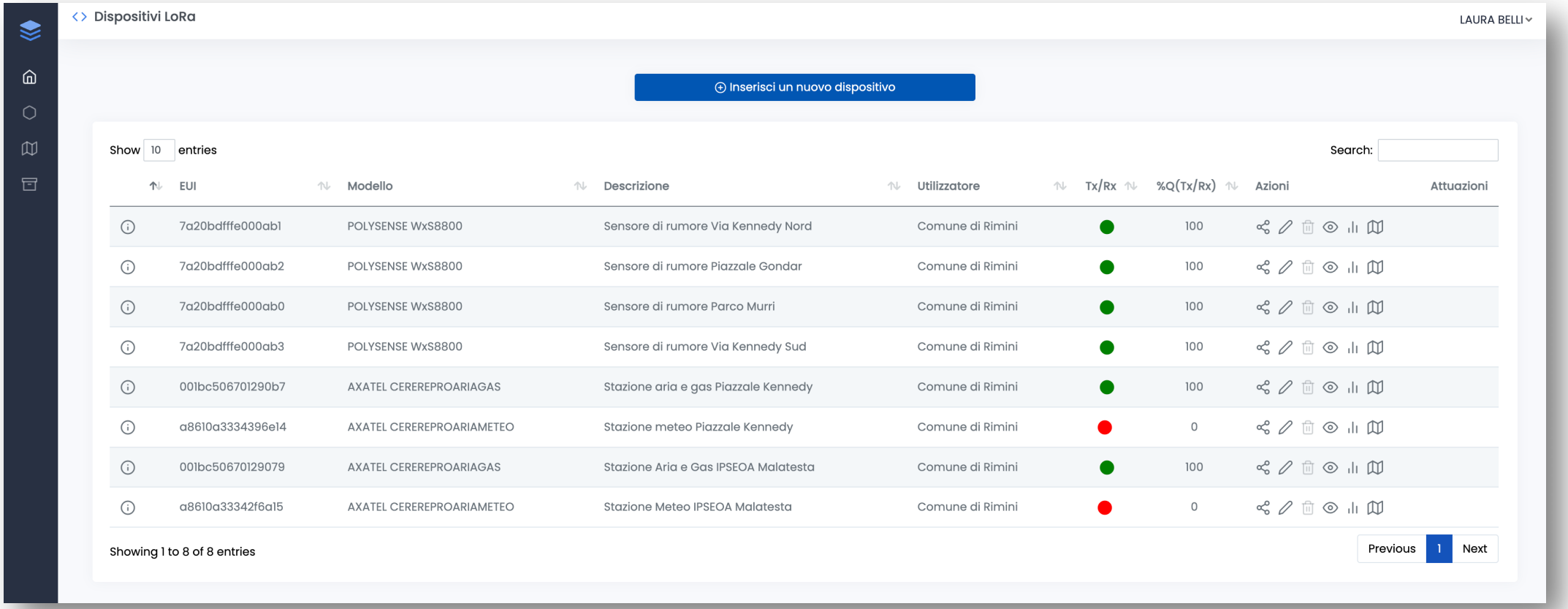


Smart city scenario: RetePAIoT

- It is a regional project developed by Lepida: a regional company working as a hub to support Emilia Romagna ICT development
- RetePAIoT aims at developing a LoRaWAN IoT network, in which citizens and companies, in addition to the Public Administration (PA) can integrate their own sensors
- It integrates several gateways in the area of the Emilia-Romagna region
- Everyone can access for free through the Italian digital identity SPID or CIE
- If you want to deploy or test a LoRaWAN device, and you don't have a gateway you can use this service
- Possibility to share devices with other people or companies
- Access to the collected data through HTTP REST API

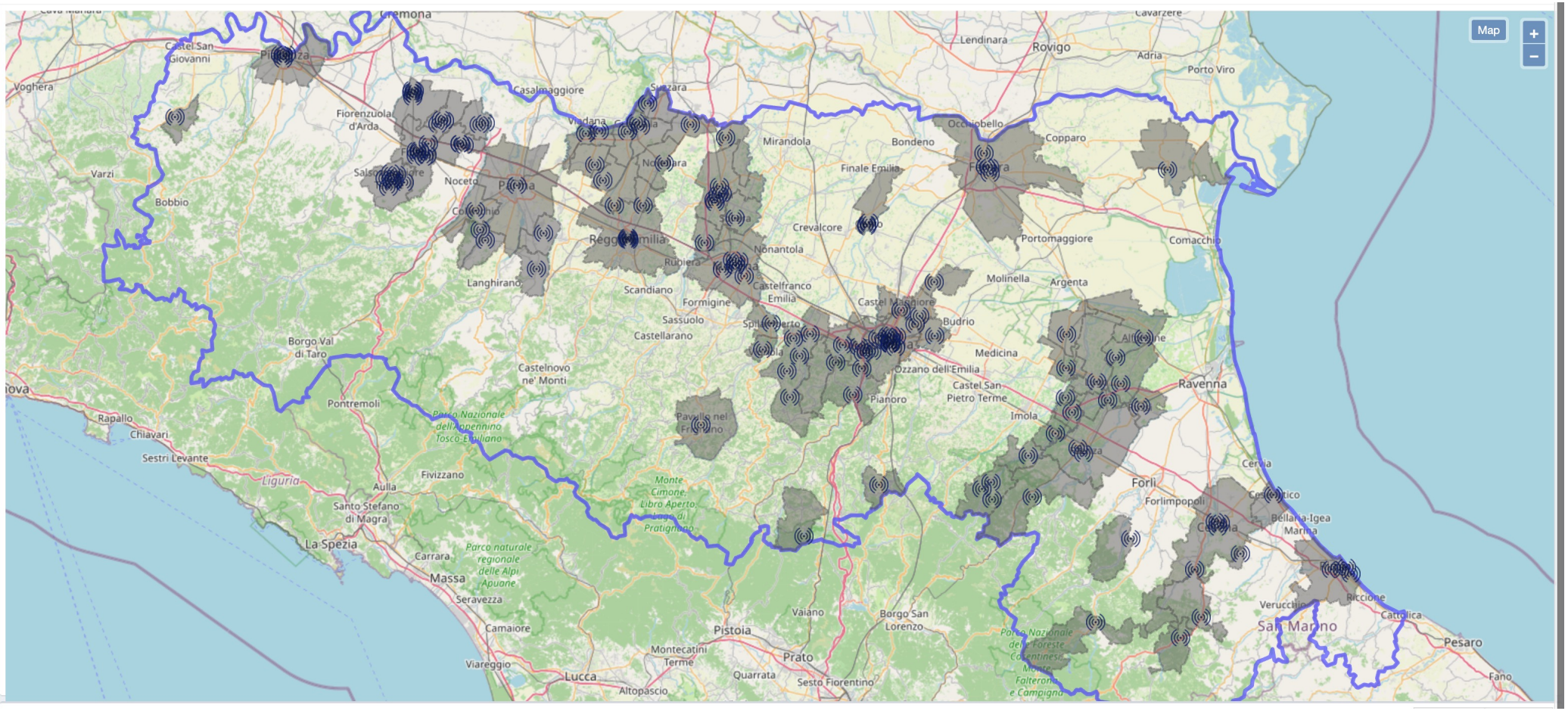
Smart city scenario: RetePaIoT

RetePAIoT dasboard



Smart city scenario: RetePaIoT

RetePAIoT covered areas and gateways



The Things Network: hands on devices

- Open the TTN Console at https://eu1.cloud.thethings.network/
- Create the “UniPR IoT Course 2026” Application
- Register a Soil Sensor (Milesight EM500-SMTC)
  - Frequency Plan: Europe 863-870 MHz (SF12 for RX2)
  - LoRaWAN Version : 1.0.3
  - AppEUI (JoinEUI): 24E124C0002A0001
  - DEVEui: 24E124126D119510
  - AppKey: generate from TTN
  - End device ID: soil-sensor-24e124126d119510
- Save the APP key on the device through the Milesight Toolbox App
- See Uplink and add the payload decoder
- https://github.com/Milesight-IoT/SensorDecoders/tree/main/em-series/em500-smtc



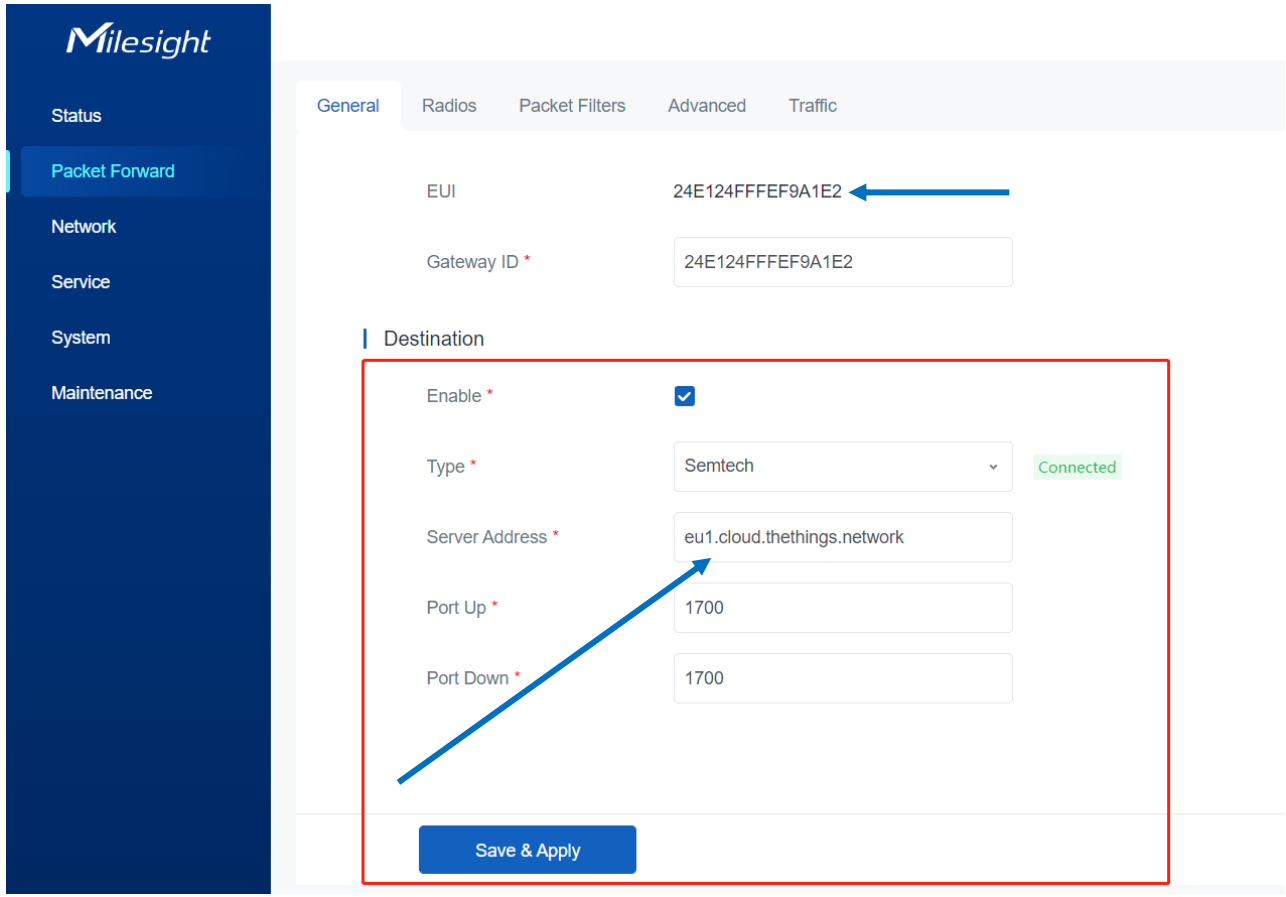
The Things Network: hands on gateways

- Open the TTN Console at
  https://eu1.cloud.thethings.network/
- Create the IoTCourse Application
- Register a new gateway (Milesight UG63)
  - Gateway EUI: 24E14FFFFEF8E338
  - Gateway ID: iot-course-2025-gw
  - Frequency Plan: Europe 863-870 MHz (SF12 for RX2)

- Change the Packet Forwarder configuration in the gateway

Gateway setup

- You need to configure the Packet Forwarder on the MileSight gateway to forward data on our TTN server
- We can do it through the web configuration dashboard provided by the gateway (the GW is connected to the Internet!)
- In this dashboard we can also see the Gateway EUI

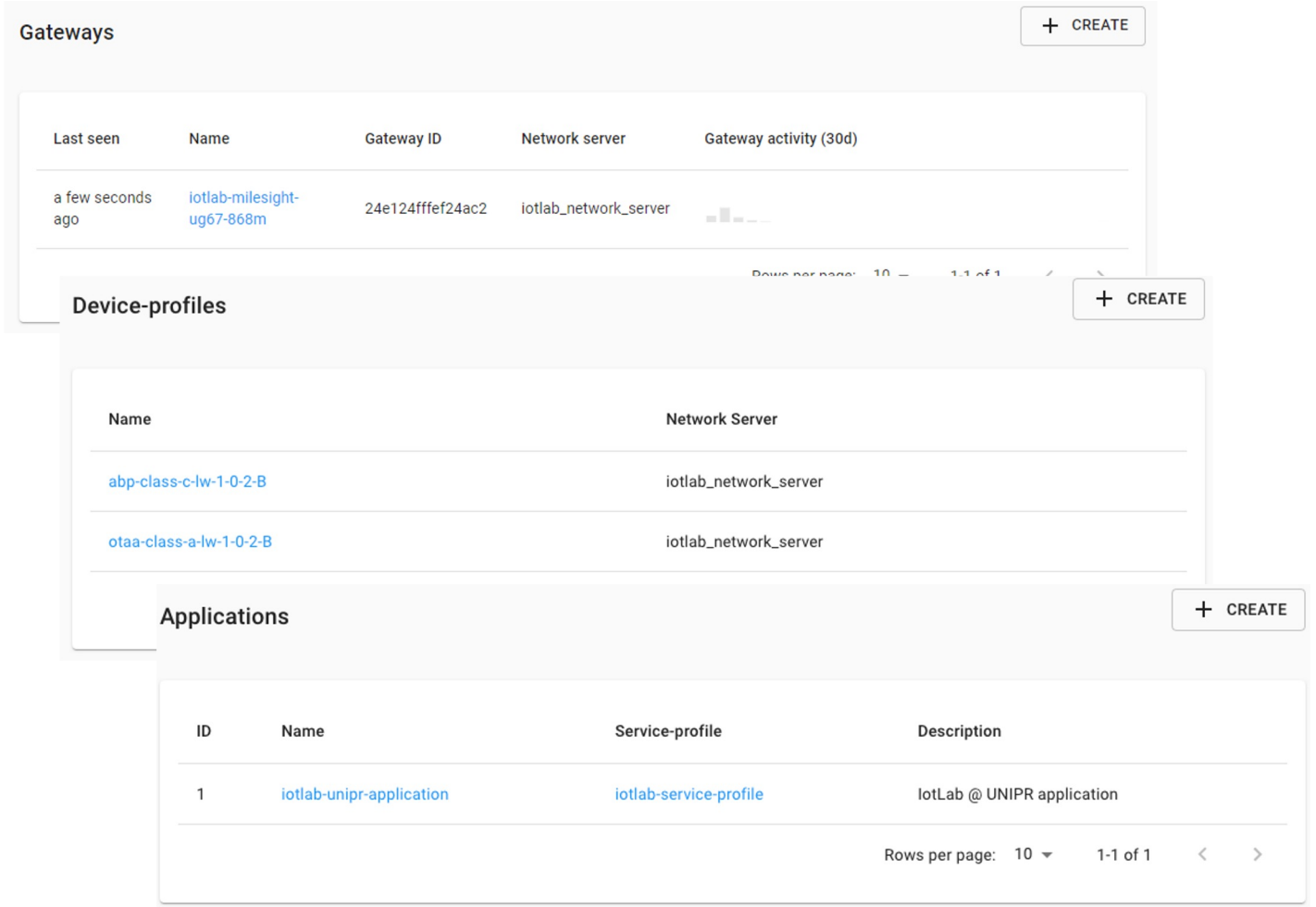


All together in a LoRaWAN Network example

- End node Alpha:
  - Microchip RN2483 LoRa Mote (support 1.0.2 LoRaWAN, regional parameter spec B)
  - Class A
  - Activation: OTAA
  - Sends temperature and light values trough on board application
- End node Beta:
  - Microchip RN2483 LoRa DAUGHTER BOARD (support 1.0.2 LoRaWAN, regional parameter spec B)
  - Class C
  - Activation: ABP
  - Sends CO_2 level
  - Accept downlinks to activate an acoustic signal if the CO_2 level is too high
- Gateway: Milesigth UG67
- Network Server and Application Server:
  - ChirpStack installed on a UNIPR host → we have admin privileges
  - Dashboard available on http://<unipr-host>:8080

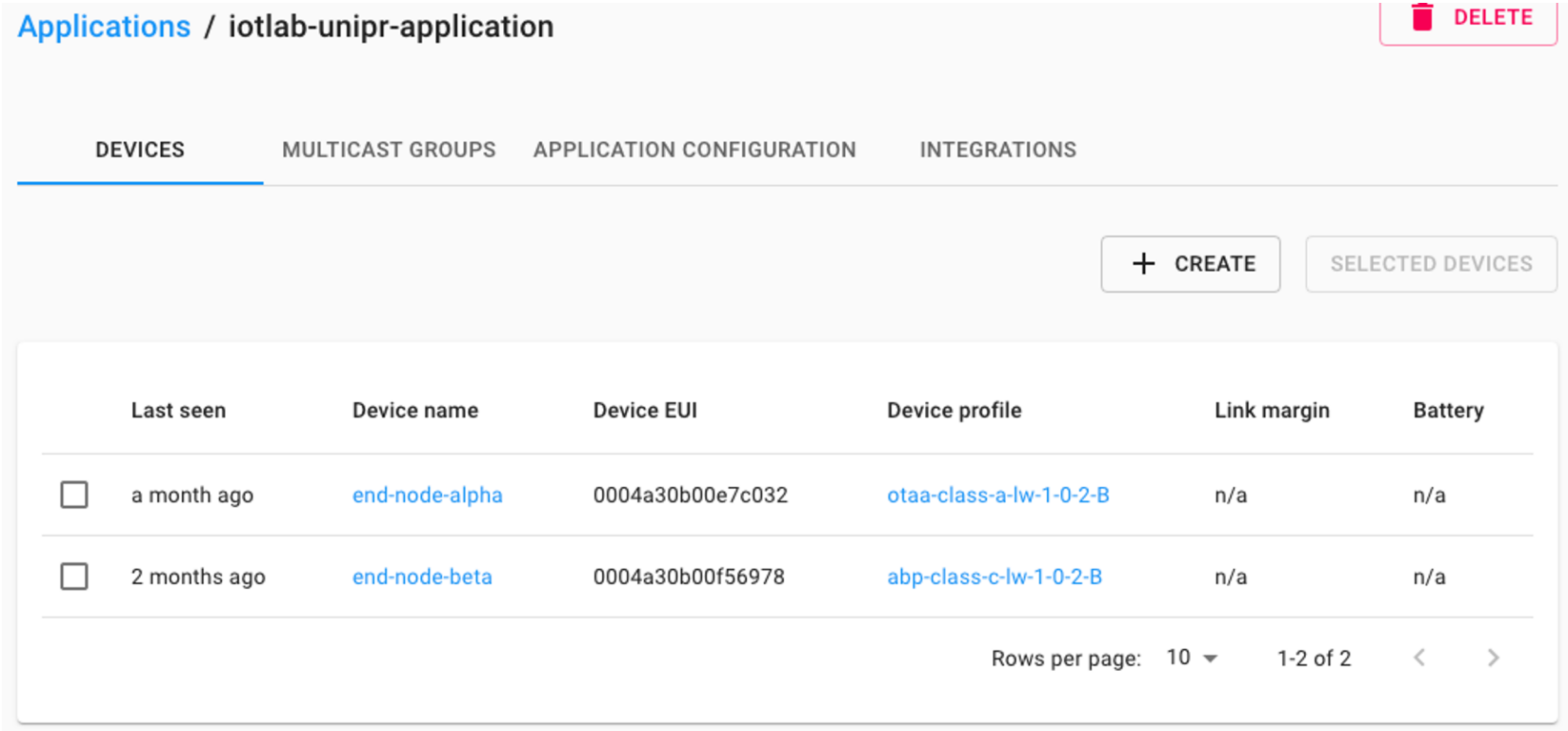
ChirpStack Entities

- Add a Tenant
- Add the Gateway (we know the unique EUI)
- Add 2 different Device Profiles
- Add an Application



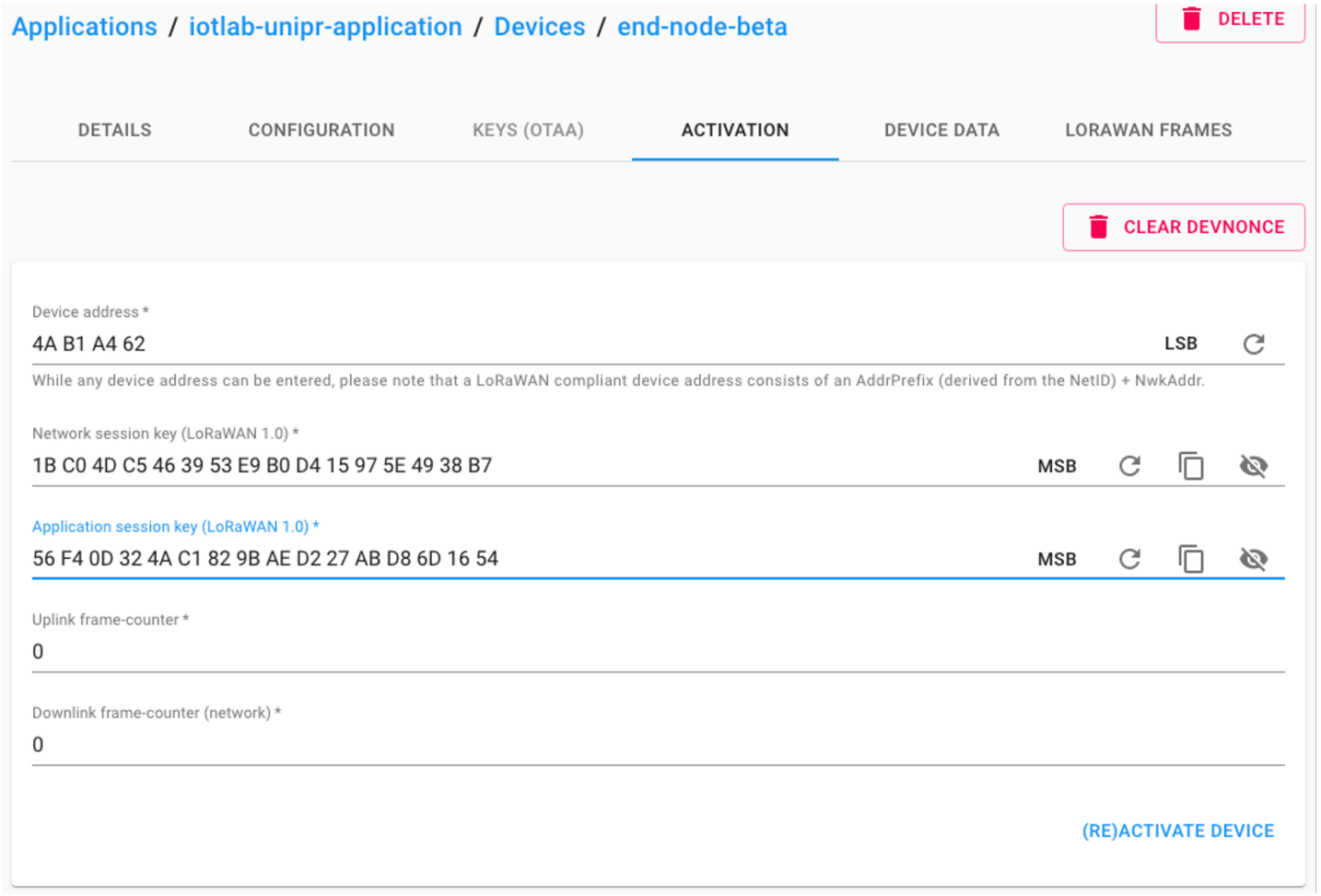
ChirpStack Entities

- Add devices Alpha and Beta
- Connect them to the correct device profile
- Add Activation parameters



ChirpStack Entities

Beta device ABP Activation parameters



Endnode Beta setup

- Connect the device to a host computer
- Open a serial connection (e.g. with Putty program)
- You can send serial commands
- Suppose the $C0_2$ level is of 735 ppm (373335 hex) and you want to send data on port 22
- If you receive downlinks, in console will appear “mac rx”
- You can do it also with a program (e.g. Python script)

```
sys get hweui //Returns the globally unique EUI for device
> 0004a30b00f56978

mac set class c
> ok

mac set adr on
> ok

mac set devaddr 4AB1A462
> ok

mac set nwkskey 1BC04DC5463953E9B0D415975E4938B7
> ok

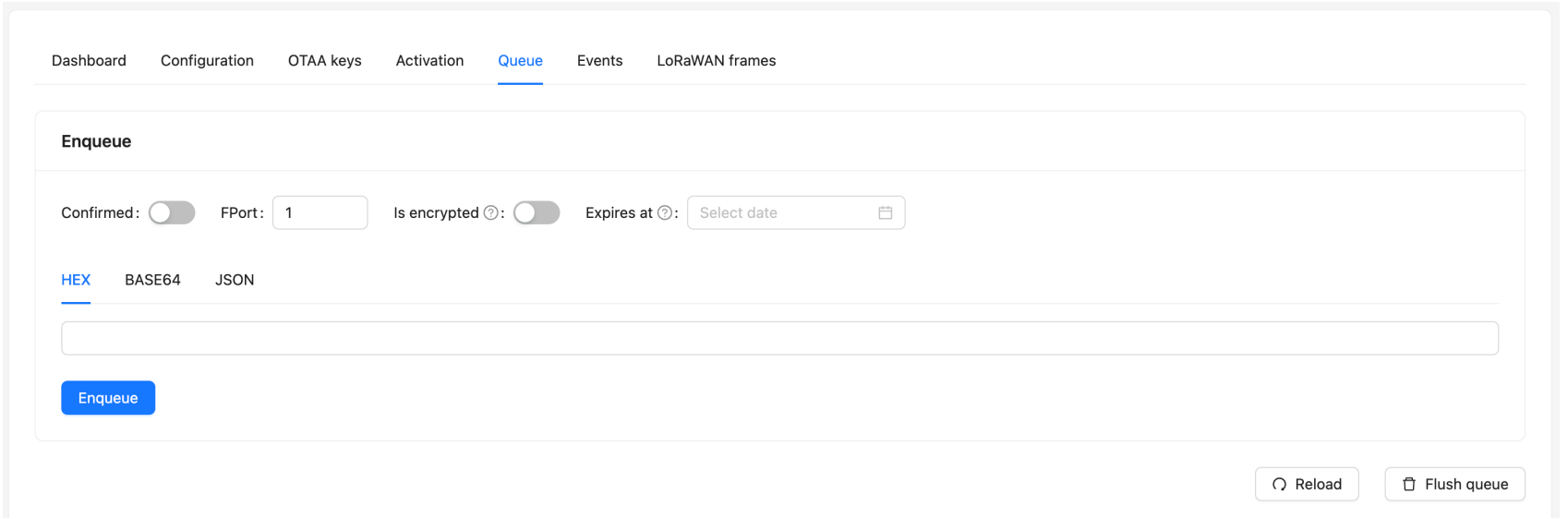
mac set appskey 56F40D324AC1829BAED227ABD86D1654
> ok

mac join abp
> join accepted

mac save
> ok
mac tx uncnf 22 373335
> ok
> mac tx ok

> mac rx 60 1
```

Enqueue of downlink messages for Beta



- Downlink messages can be enqueued through the ChirpStack device page
- Parameters:
  - FPort: eg: 60
  - type: confirmed or not
  - content: base64 encoded
- message 1 → MQ==

If you want to know more…

- https://www.thethingsnetwork.org/
- https://www.chirpstack.io/
- https://www.milesight-iot.com/lorawan/gateway/ug67/
- https://www.microchip.com/en-us/product/RN2483
- https://github.com/Lora-net/packet_forwarder
- …

LoRaWAN technology: An overview



IoT Lab

UNIVERSITÀ DI PARMA

April 10, 2025

Internet of Things, Communication Engineering Course